

BUILD. CONNECT. DEPLOY.
FROM ANDROID TO BACKEND

Android Full-stack Developer

دليل عملي لبناء تطبيقات أندرويد متكاملة
من الواجهة الأمامية إلى الخادم وقواعد البيانات



Android App
Development



RESTful APIs &
Backend Development



Database Design &
Management



Cloud Deployment
& DevOps



تأليف
هيثم زيدان



التاريخ
08 July 2026

ملاحظة هامة:

بعض الأكواد في هذا الكتاب، خصوصا الطويلة منها غير مطبوعة في الكتاب و مرقمة بالتسلسل أثناء الشرح. تبدأ ب Code 1, Code 2 و هكذا.

للحصول على الأكواد عليك شراء الكتاب على الرابط التالي

<https://www.patreon.com/posts/163270057>

للإستفسار عبر البريد الإلكتروني

هيثم زيدان

haythmzeidan@gmail.com

بسم الله الرحمن الرحيم

مقدمة:

نحو آفاق جديدة في تطوير الأندرويد...

في عالم يتسارع فيه التحول الرقمي، لم يعد تطوير تطبيقات الأندرويد مجرد كتابة أكواد برمجية على واجهات المستخدم فحسب، بل أصبح رحلة متكاملة تبدأ من الفكرة وتمر عبر هندسة السيرفرات (Backend)، وصولاً إلى تقديم تجربة مستخدم (UI/UX) استثنائية. إن مفهوم "Full-stack Mobile Developer" الذي أتينا في هذا الدليل هو الجسر الذي يعبر بك من مرحلة "المبرمج الذي يكتفي ببناء الواجهات" إلى "المهندس الذي يمتلك النظام بأكمله". بصفتك مطوراً يطمح للتميز في سوق العمل المحلي أو الدولي، فإن هذا الدليل ليس مجرد دليل تقني للمبتدئين، بل هو خارطة طريق احترافية تم تصميمها لتمكينك من بناء تطبيقات متكاملة وقابلة للتوسع. لقد قمت هنا بدمج أدوات أندرويد الحديثة (Jetpack Compose, Room, Retrofit, Hilt) مع تقنيات السيرفرات المرنة (Node.js) لأضع بين يديك أسرار النظام الذي يضمن أداءً عالياً، وتجربة أوفلاين-أول (Offline-First) سلسلة، ومعايير أمنية تضاهي أكبر التطبيقات العالمية. إن هذا العمل موجه إليك، ليس فقط لتتعلم "كيف" تكتب الكود، بل لتفهم "لماذا" يتم بناؤه بهذه الطريقة، وكيف تحول مشروعك من مجرد تطبيق تقني إلى منتج تجاري ملموس قادر على المنافسة في المتاجر الرقمية. سواء كنت تسعى لتطوير محفظة أعمالك البرمجية لتستقطب بها كبرى الشركات، أو كنت تخطط لتأسيس وكالتك الخاصة لتقديم خدمات تقنية متكاملة، فإن هذا الكتاب سيكون رفيقك في كل خطوة—من كتابة السطر الأول في `server.js` وحتى اللحظة التي تضغط فيها زر الإطلاق النهائي على [Google Play Console](#). استعد لخوض تجربة عملية مكثفة، تتجاوز النظريات لتلامس واقع المشاريع البرمجية الحقيقية. مرحباً بك في عالم المطور المتكامل.

المؤلف

هيثم زيدان

08 July 2026

Android (منهج تطوير تطبيقات أندرويد المتكامل Full-stack Developer)

هذا الفهرس يغطي جميع المراحل اللازمة لبناء تطبيقات أندرويد احترافية، بدءاً من إعداد البيئة وحتى نشر التطبيق.

(Foundation) المرحلة الأولى: الأساسيات والبيئة

قبل البدء بالبرمجة، يجب إعداد الأدوات وفهم الأساسيات.

الموضوع	التفاصيل
إعداد بيئة العمل	فهم واجهة البرنامج، وإعداد Android Studio، تحميل وتنصيب أو ربط جهازك الحقيقي (Emulator) محاكي.
(Kotlin) لغة البرمجة	(وهي اللغة الرسمية والمدعومة من جوجل) Kotlin تعلم لغة. - المتغيرات، الأنواع، العمليات الحسابية. - هيكل التحكم (if, when, loops). (OOP) والبرمجة الكائنية (Functions) الدوال.
مفاهيم أندرويد الأساسية	- Activity؟ ما هو الـ (Activity Lifecycle) دورة حياة التطبيق. - AndroidManifest.xml فهم ملف.

(UI/UX) المرحلة الثانية: تصميم واجهات المستخدم

هنا سنتنقل من الكود إلى بناء الشكل الذي يراه المستخدم.

الموضوع	التفاصيل
Jetpack Compose	هو الإطار الحديث والموصى به حالياً من جوجل لبناء الواجهات. - Composables التعرف على. - ترتيب العناصر (Column, Row, Box). - و (State Management) التعامل مع الحالات (Remember).
(للمعرفة فقط) XML	التقليدية Layouts يفضل أخذ نظرة سريعة على الـ (ConstraintLayout) لأنك قد تجدها في مشاريع قديمة.

(Data & Networking) المرحلة الثالثة: الربط والبيانات

التطبيق الذي لا يتصل بالإنترنت أو لا يخزن بيانات يعتبر محدوداً.

الموضوع	التفاصيل
(Navigation) التنقل	Navigation Compose كيفية التنقل بين الشاشات باستخدام.
(Networking) الشبكات	APIs للتعامل مع الـ Retrofit استخدام مكتبة أو Kotlin Serialization تحويل البيانات باستخدام Gson.
قواعد البيانات المحلية	لتخزين البيانات محلياً على الجهاز Room استخدام مكتبة.

الموضوع	التفاصيل
المهام غير المتزامنة	وهي ضرورية جداً لأي مطور Flow و Coroutines تعلم (أندرويد محترف).

(Architecture) المرحلة الرابعة: الهندسة المعمارية

كيف تبني تطبيقاً قابلاً للصيانة والتطوير؟

الموضوع	التفاصيل
MVVM (Model-View-ViewModel)	النمط القياسي لتنظيم كود أندرويد.
Dependency Injection	لتسهيل إدارة تبعيات الكود Hilt تعلم استخدام مكتبة.
Jetpack Libraries	ViewModel, LiveData, DataStore التعرف على مكتبات مثل.

المرحلة الخامسة: الاحتراف والنشر

الموضوع	التفاصيل
التصحيح (Debugging)	Android Studio وأدوات التحليل في Logcat استخدام.
الاختبار (Testing)	(Unit Testing) كتابة اختبارات للوحدات.
النشر	(Signing the App) تجهيز التطبيق للرفع. - Google Play Console رفع التطبيق على.

Android Full Stack Developer

إعداد بيئة العمل

تحميل وتثبيت Android Studio، فهم واجهة البرنامج، وإعداد محاكي (Emulator) أو ربط جهازك الحقيقي. إعداد بيئة العمل هي حجر الأساس، إعداد بيئة عمل سليمة سيجنبك الكثير من المشاكل البرمجية لاحقاً. سنقوم بتثبيت Android Studio، وهو البرنامج المتكامل (IDE) الذي ستستخدمه لكتابة الكود، تصميم الواجهات، وتشغيل التطبيقات.

إليك الخطوات التفصيلية بالترتيب:

1. المتطلبات التقنية للجهاز (Hardware Requirements)

قبل التحميل، تأكد أن جهازك يلبي المواصفات الدنيا ليعمل البرنامج بسلاسة:

- **نظام التشغيل:** Linux، أو macOS، Windows (10/11).
- **الذاكرة العشوائية (RAM):** 8 جيجابايت كحد أدنى (يُفضل بشدة 16 جيجابايت لأن Android Studio يستهلك الكثير من الذاكرة).
- **المعالج:** أي معالج حديث (Intel Core i5 أو ما يعادله من AMD/Apple Silicon).
- **المساحة التخزينية:** 8 جيجابايت على الأقل (يفضل توفر مساحة فارغة على قرص SSD لسرعة التشغيل).

2. تحميل وتثبيت Android Studio

1. اذهب إلى الموقع الرسمي: developer.android.com/studio.
2. اضغط على زر Download Android Studio.
3. وافق على الشروط والأحكام وابدأ التحميل.
4. بعد انتهاء التحميل، قم بفتح ملف التثبيت:
 - في Windows: اتبع معالج التثبيت (Next -> Next) وتأكد من اختيار Android Virtual Device ضمن المكونات المختارة.
 - في macOS: اسحب أيقونة Android Studio إلى مجلد Applications.

3. إعداد البرنامج لأول مرة (Setup Wizard)

عند فتح البرنامج لأول مرة، سيظهر لك "معالج الإعدادات":

- **(أ) Standard Setup** اختر الإعداد القياسي Standard، فهو يضبط كل ما تحتاجه تلقائياً.
- **(ب) Theme** اختر النمط الذي تفضله (Dark/Light).
- **(ج) SDK Components** سيقوم البرنامج بتحميل ملفات الـ Android SDK (وهي المكتبات التي يمكنك من كتابة تطبيقات لأجهزة أندرويد) وأدوات البناء. اترك هذه العملية تكمل حتى النهاية.

4. إعداد المحاكى (Virtual Device)

بدلاً من استخدام هاتفك الشخصي في البداية، سننشئ هاتفاً وهمياً (Emulator):

1. من واجهة Android Studio، اختر Virtual Device Manager (أو ابحث عن أيقونة الهاتف).
2. اضغط على Create Device.
3. اختر فئة (مثل Phone) ثم اختر طرازاً (مثلاً Pixel 6 أو Pixel 7).
4. اختر نسخة نظام أندرويد (يُفضل تحميل أحدث إصدار متاح، مثلاً Android 14 أو 15).
5. أعطِ المحاكى اسماً واضغط Finish. الآن لديك هاتف داخل جهاز الكمبيوتر.

5. التحقق من التثبيت (إنشاء أول مشروع)

لنكتب أول كود لنا للتأكد أن كل شيء يعمل:

1. في الصفحة الرئيسية للبرنامج، اختر New Project.
2. اختر قالب Empty Activity.
3. ضع اسماً لمشروعك (مثلاً: MyFirstApp).
4. تأكد أن اللغة المحددة هي Kotlin.

اضغط Finish

ستظهر لك نافذة التحميل (Gradle Sync) في الأسفل. انتظر حتى ينتهي البرنامج من تحميل المكتبات (يستغرق ذلك وقتاً طويلاً في المرة الأولى فقط، يجب أن تكون متصلاً بالإنترنت). بمجرد انتهاء التحميل، اضغط على زر Run (الأيقونة الخضراء المثلثة) وسيعمل المحاكى ليظهر لك عبارة "Hello Android!".

نصيحة إضافية:

إذا كان جهازك ضعيف الإمكانيات، أنصحك بشدة باستخدام هاتفك الحقيقي بدلاً من المحاكى.

- اذهب إلى إعدادات هاتفك -> "حول الهاتف" -> اضغط 7 مرات متتالية على "رقم الإصدار" (Build) سيظهر لك خيار خيارات المطور أسفل حول (Developer Options) لتفعيل خيارات المطور (Number) الهاتف. انقره عليه ثم
 - ادخل على خيارات المطور وفعل تصحيح أخطاء (USB Debugging) (USB).
 - صل هاتفك بالكمبيوتر، وسيتعرف عليه Android Studio تلقائياً كجهاز للتجارب.
- ممتاز، بما أنك قمت بإعداد البيئة، فلنلقِ نظرة على "المطبخ" الذي ستعمل فيه. عندما تفتح مشروعك في على الجانب الأيسر (Project Tool Window) ستجد نافذة المشروع، Android Studio، فهم الملفات هو نصف الطريق للاحتراف. إليك أهم المجلدات التي ستتعامل معها يومياً:

1. مجلد app

هذا هو المجلد الرئيسي الذي يحتوي على كل ما يتعلق بتطبيقك.

- **manifests:** يحتوي على ملف `AndroidManifest.xml`. هذا هو "بطاقة الهوية" للتطبيق؛ فيه تُحدد اسم التطبيق، الأيقونة، والأهم من ذلك: الصلاحيات (مثل الوصول للإنترنت، الكاميرا، أو الموقع الجغرافي).
- **(أو kotlin أو java):** هنا يقع قلب التطبيق. ستجد ملفات الكود التي تنتهي بـ `.kt`. في المشاريع الحديثة التي تستخدم Jetpack Compose، ستكتب معظم منطق الواجهة هنا أيضاً.
- **(Resources - الموارد) res:**
 - الصور، الأيقونات، وأشكال الخلفيات: `drawable`.
 - لتعريف واجهات المستخدم (XML) في حال كنت تستخدم `layout`.
 - `themes.xml` و (للألوان) `colors.xml` و (للنصوص) `strings.xml` يحتوي على ملفات مثل `values` (لشكل التطبيق). هذه تجعل تعديل الألوان والنصوص في كامل التطبيق يتم من مكان واحد.

2. ملفات الـ Gradle (الأهم على الإطلاق)

إذا كنت تتساءل كيف يفهم Android Studio المكتبات التي تطلبها، فالسر في الـ Gradle. ستجد ملفين باسم `build.gradle.kts`:

- **Project Level:** يتحكم في إعدادات المشروع ككل.
- **Module Level (app):** هذا الملف هو الذي تضع فيه المكتبات الخارجية (مثل Retrofit للإنترنت أو داخل `dependencies` لقواعد البيانات). أي مكتبة جديدة تريد استخدامها، ستضيفها في قسم `Room` هذا الملف.

3. ملف MainActivity.kt

هو نقطة الدخول (Entry Point) للتطبيق. أي تطبيق أندرويد يحتاج إلى "نشاط" (Activity) واحد على الأقل ليبدأ. ستجد فيه عادةً دالة تسمى onCreate؛ هذا هو المكان الذي يُخبر التطبيق فيه: "ابدأ برسم واجهة المستخدم عند فتح البرنامج".

💡 تجربة عملية صغيرة:

لنتأكد من أنك تتقن التنقل في ملفاتك، جرب القيام بالتالي الآن:

1. افتح ملف res/values/strings.xml.
 2. قم بتغيير النص الموجود بين وسمي <string name="app_name">...</string> إلى اسم مشروعك باللغة العربية.
 3. افتح ملف MainActivity.kt وشاهد كيف يتم استدعاء واجهة المستخدم.
- هل تظهر لك أي تحذيرات (Warnings) في ملف الـ Gradle أو هل كل شيء يبدو هادئاً (بدون خطوط حمراء)؟
- إذا لا، هذا خبر ممتاز! عدم وجود خطوط حمراء يعني أن Gradle قام ببناء المشروع بشكل سليم وتعرف على كافة المكتبات الأساسية، وهذا يعني أنك جاهز تماماً للبدء في كتابة الكود.
- بما أن البيئة جاهزة والمشروع سليم، دعنا ننتقل إلى الخطوة التالية التي تضعك على طريق "المطور الحقيقي": هيكل الكود (Code Structure).
- بما أننا نستخدم Kotlin و Jetpack Compose (التقنية الحديثة)، فأنت لا تحتاج للتعامل مع ملفات XML المعقدة للواجهات. سنركز على "الدوال القابلة للتكوين" (Composable Functions).

تحدي البداية: فهم الـ Composable

افتح ملف MainActivity.kt. ستجد دالة غالباً باسم Greeting أو محتوى مشابه. جرب تعديل الكود ليعرض اسمك بدلاً من "Android":

ملاحظة هامة: الأكواد في هذا الكورس خصوصاً الطويلة منها، محفوظة بملفات نصية و مرقمه حسب تسلسل الكود في الكورس، مثلاً إذا أردت نسخ كود معين من ملف نصي ستجده في الملفات المرفقة عند شراء هذا الكورس.

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "في تطبيقك الأول Haytham مرحباً بك يا ",
        modifier = modifier
    )
}
```

و هذا هو أول كود في الكورس رقم الكود 1 Code

ما الذي حدث هنا؟

- **@Composable**: هو "وسم" (Annotation) يخبر Android Studio أن هذه الدالة مسؤولة عن رسم واجهة مستخدم.
- **Text(...)**: هو عنصر واجهة (UI Component) جاهز توفره مكتبة Compose لعرض النصوص.

(أساسيات لغة Kotlin):

التمكن من Kotlin سيجعل تصميم الواجهات والتعامل مع البيانات لاحقاً أمراً في غاية السهولة. لغة Kotlin مصممة لتكون "مختصرة وآمنة"، وهي اللغة المفضلة حالياً لتطوير أندرويد.

سنبدأ بـ الأساسيات الأربعة التي ستحتاجها لبناء أي تطبيق:

1. المتغيرات (Variables)

في Kotlin، نستخدم كلمتين مفتاحيتين لتعريف المتغيرات:

- **val**: للمتغيرات التي لا تتغير قيمتها لاحقاً (ثابتة).
- **var**: للمتغيرات التي تتغير قيمتها.

```
val name = "Haytham" // لا يمكن تغيير الاسم لاحقاً
var age = 30 // يمكن تغيير العمر لاحقاً (مثلاً عند حلول عيد الميلاد)
age = 31 // هذا صحيح برمجيّاً
```

2. أنواع البيانات (Data Types)

لغة ذكية، Kotlin تعرف نوع البيانات تلقائياً، ولكن من الجيد معرفة الأنواع الأساسية:

- **String** للنصوص (مثلاً: "Android")
- **Int** للأرقام الصحيحة (مثلاً: 2026)
- **Double** للأرقام العشرية (مثلاً: 99.9)
- **Boolean** للقيم المنطقية (true أو false)

3. الدوال (Functions)

الدالة هي كتلة كود تقوم بمهمة محددة. لاحظ كيف نكتبها في Kotlin:

```
fun greetUser(name: String): String {  
    return "مرحباً يا $name"  
}  
// يمكنك استدعاؤها هكذا:  
val message = greetUser("Haytham")
```

ملاحظة: علامة \$ داخل النص (String Template) تسمح لنا بإدراج قيمة المتغير مباشرة داخل النص.

4. التحكم في التدفق (Control Flow)

كيف تجعل التطبيق يتخذ قرارات؟

- **if / else:**

```
val isOnline = true  
if (isOnline) {  
    println("المستخدم متصل")  
} else {  
    println("المستخدم غير متصل")  
}
```

- **when (في اللغات الأخرى switch بديل قوي لـ):**

```
val status = 200
when (status) {
    200 -> println("نجاح")
    404 -> println("غير موجود")
    else -> println("خطأ غير معروف")
}
```

🔗 تحدي عملي سريع (الك):

في MainActivity.kt (أو يمكنك تجربة ذلك في Kotlin Playground):
محاكي ويب للغة <https://play.kotlinlang.org>

قم بتعريف متغيرين: واحد لاسمك (val name)، وواحد لعدد سنوات خبرتك التقنية (var years). ثم اكتب دالة بسيطة تأخذ هذه المتغيرات وتطبع جملة مثل: "أنا Haytham ولدي X سنوات من الخبرة".

إذا لم تواجه صعوبة في كتابة الكود فهذا رائع! هذا يعني أنك أصبحت الآن قادراً على كتابة المنطق الأساسي لأي تطبيق.

و بما أننا تجاوزنا مرحلة المتغيرات والدوال، دعنا ننتقل إلى "مفاهيم Kotlin المتقدمة قليلاً" والتي ستصادفها في كل مشروع أندرويد حقيقي:

1. التعامل مع القيم الفارغة (Null Safety)

في لغات أخرى، الخطأ الأكثر شيوعاً هو NullPointerException. في Kotlin، اللغة مصممة لمنع حدوث هذا الخطأ.

- المتغير العادي لا يمكن أن يكون فارغاً.
- إذا كنت تتوقع أن المتغير قد يكون فارغاً، يجب إضافة علامة استفهام ? بعد نوع البيانات:

```
var username: String? = null // هذا مسموح به
```

2. الكلاسات (Classes)

أنت كمطور تعرف أن البرمجة الكائنية (OOP) هي العمود الفقري لأي نظام.

```
class Person(val name: String, val age: Int) {
    fun introduce() {
        println("اسمي $name وعمرى $age")
    }
}
// استخدام الكلاس:
val haytham = Person("Haytham", 30)
haytham.introduce()
```

3. التجميعات (Collections)

كيف تخزن قائمة من العناصر؟ (مثل قائمة تطبيقات أو قائمة أسماء):

```
val list = listOf("Android", "iOS", "Flutter") // قائمة ثابتة
val mutableList = mutableListOf("WordPress", "Laravel") // قائمة يمكن التعديل
عليها
mutableList.add("React Native")
```

تمرين:

لنبدأ في بناء "كلاس" بسيط يمثل كيان بيانات (Data Class) (مثلاً: كلاس يمثل "مهمة" To-Do أو "منتج" في متجر)، لترى كيف نربط البيانات بالواجهة لاحقاً.

يُستخدم الـ data class خصيصاً لحمل البيانات (Data holders)، وهو يقوم بإنشاء الدوال الضرورية مثل `toString()` و `equals()` تلقائياً.

لنفترض أنك تبني تطبيقاً لمهامك اليومية (To-Do App). سننشئ كلاس يمثل "المهمة":

1. تعريف الـ Data Class

أضف هذا الكود في ملف جديد أو داخل `MainActivity.kt`:

```
data class TodoItem(
    val id: Int,
    val title: String,
    val isCompleted: Boolean = false // القيمة الافتراضية للبدء
)
```

2. إنشاء قائمة بيانات

الآن لنقم بإنشاء قائمة تحتوي على عدة مهام برمجية:

```
val myTasks = listOf(  
    TodoItem(1, "دراسة Jetpack Compose"),  
    TodoItem(2, "تطوير كود Laravel"),  
    TodoItem(3, "صيانة أجهزة CCTV")  
)
```

3. الربط مع الواجهة

في Jetpack Compose، نقوم بتحويل هذه البيانات إلى واجهة مستخدم باستخدام عنصر يسمى LazyColumn (وهو يشبه RecyclerView).

سيبدو الكود الذي يعرض هذه القائمة في تطبيقك هكذا:

```
@Composable  
fun TaskList(tasks: List<TodoItem>) {  
    LazyColumn {  
        items(tasks) { task ->  
            Text(text = "المهمة: ${task.title}")  
        }  
    }  
}
```

تحدي عملي لك الآن:

أريدك أن تقوم بما يلي:

1. قم بتعريف data class يمثل منتجاً (Product) في متجر إلكتروني (يجب أن يحتوي على: name ك String، و price ك Double).
2. قم بإنشاء قائمة من 3 منتجات باستخدام listOf.
3. حاول تشغيل البرنامج ورؤية كيف يتم ترتيب البيانات.

لننتقل الآن من التخطيط إلى التنفيذ الفعلي. سنقوم بدمج الـ data class الذي قمنا بتعريفه داخل واجهة المستخدم.

سنقوم بتعديل دالة Greeting في MainActivity.kt (أو إنشاء دالة جديدة) لتعرض قائمة المنتجات التي أنشأتها.

الكود التطبيقي:

استبدل المحتوى في ملف MainActivity.kt بهذا الكود البسيط، وراقب كيف يتم تحويل البيانات إلى واجهة:

رقم الكود 2 Code

شرح الكود:

- **LazyColumn:** يعمل مثل "قائمة مرنة" تقوم بعرض العناصر واحداً تلو الآخر. إذا كان لديك 1000 منتج، سيعرض فقط ما يراه المستخدم على الشاشة ليوفر في استهلاك الذاكرة.
- **items(products):** هذه الدالة تقوم بعمل "دورة" (Loop) على قائمتك وتُنشئ عنصراً (Text) لكل منتج داخل القائمة.
- **Modifier.padding:** أضفنا مسافة بسيطة حول القائمة لتبدو بشكل أنيق.

الآن دورك:

قم بنسخ هذا الكود واستبدال محتوى ملفك الحالي به، ثم قم بتشغيل التطبيق على المحاكى أو هاتفك الحقيقي. هذه هي الخطوة التي تحول التطبيق من مجرد "شاشة عرض" إلى "تطبيق تفاعلي".

سنقوم الآن بتعديل دالة عرض المنتج لتصبح قابلة للضغط باستخدام clickable وإضافة "زر" بسيط، وسنستخدم Log.d لطباعة النتيجة في Logcat (السجل الخاص بالمبرمجين).

الكود المعدل:

استبدل دالة ProductList بهذا الكود، ولا تنسَ إضافة `import android.util.Log` في أعلى الملف:

رقم الكود 3 code

كيف نتحقق من النتيجة؟

1. شغل التطبيق على المحاكى أو هاتفك.
2. في Android Studio، ابحث عن تبويب Logcat في الجزء السفلي من البرنامج.

3. في مربع البحث داخل Logcat، اكتب AppAction.
4. اضغط على أحد المنتجات في تطبيقك، وستشاهد فوراً ظهور الرسالة: لقد ضغطت على: ... في نافذة Logcat.

الآن بعد أن عرفنا كيف نكتشف "الضغط"، هل يمكنك محاولة إضافة "زر" (Button) حقيقي بدلاً من جعل النص نفسه قابلاً للضغط؟

إليك تلميح بسيط:
في Compose، نستخدم هذا العنصر:

```
Button(onClick = { /* هنا Log ضع كود */ }) {
    Text("شراء")
}
```

هل نجحت في إضافة الزر بجانب اسم المنتج؟ (تلميح: ستحتاج لوضع الـ Text والـ Button داخل Row ليظهر بجانب بعضهما).

هذا يعني أنك بدأت تلمس قوة Jetpack Compose في بناء واجهات تفاعلية ببضع أسطر فقط.

الآن، لنضع اللمسة الأخيرة على واجهة المنتج لنرى كيف يتم تنسيق العناصر بجانب بعضها باستخدام Row (لترتيب العناصر أفقياً):

الكود المحدث (استخدام Row لدمج النص والزر):

رقم الكود 4 Code

ما الجديد هنا؟

- Row: هذا الحاوية تخبر أندرويد: "ضع العناصر بداخلي واحداً تلو الآخر في صف أفقي".
- Modifier.weight(1f): هذه إضافة عبقرية! تعطي النص مساحة "تتمدد" لتأخذ كل المساحة المتاحة، مما يدفع الزر دائماً إلى أقصى اليمين بشكل منظم.
- verticalAlignment: تضمن أن الزر والنص في منتصف الصف عمودياً.

هكذا نكون قد غطينا الأساسيات المتينة:

- إعداد البيئة والتحقق من المشروع.
- فهم لغة Kotlin (المتغيرات، الدوال، الكلاسات).

- بناء الواجهات التفاعلية (Compose) والتعامل مع القوائم.
- التعامل مع الأحداث (Click Events).

أنت الآن جاهز للانتقال لمرحلة التطبيقات الحقيقية. وسنتقل إلى المسارات التالية:

1. الربط بالإنترنت: كيف نجعل التطبيق يجلب بيانات من سيرفر حقيقي باستخدام Retrofit (مثل جلب أسعار عملات أو بيانات طقس).
2. التخزين المحلي: كيف نحفظ البيانات داخل الهاتف باستخدام Room (ليظل التطبيق يعمل حتى بدون إنترنت).
3. التنقل: كيف تصمم تطبيقاً بـ "صفحات متعددة" وتنقل بينها.

الربط بالإنترنت (Networking)

سنبدأ الآن بـ الربط بالإنترنت (Networking)، وهو الجزء الأكثر إثارة، حيث سيصبح تطبيقك "حياً" ويتواصل مع العالم الخارجي.

1. كيف يعمل الربط بالإنترنت؟

في أندرويد، نستخدم مكتبة Retrofit لأنها الأكثر استقراراً وقوة. للتعامل مع البيانات القادمة من الإنترنت (التي تأتي عادة بصيغة JSON)، سنستخدم مكتبة تحويل تسمى Kotlin Serialization.

2. إعداد المكتبات (أول خطوة)

افتح ملف build.gradle.kts (المستوى الخاص بـ app) وأضف هذه الأسطر في قسم dependencies ثم اضغط على زر Sync Now الذي سيظهر في الأعلى:

```
dependencies {
    // مكتبة Retrofit للتعامل مع الإنترنت
    implementation("com.squareup.retrofit2:retrofit:2.9.0")
    implementation("com.squareup.retrofit2:converter-gson:2.9.0") // لتحويل
    Kotlin إلى كائنات JSON
    // لإضافة صلاحية الوصول للإنترنت، افتح ملف AndroidManifest.xml
}
```

3. الصلاحية (مهم جداً)

بدون هذه الصلاحية، سيغلق التطبيق فوراً عند محاولة الاتصال. افتح ملف AndroidManifest.xml وأضف هذا السطر قبل وسم <application>:

```
<uses-permission android:name="android.permission.INTERNET" />
```

4. إنشاء واجهة التعامل (API Service)

نحن بحاجة لتعريف "عقد" يحدد كيف سيتحدث تطبيقنا مع السيرفر. لنفترض أننا نريد جلب قائمة منتجات من سيرفر تجريبي:

```
import retrofit2.http.GET

interface ApiService {
    @GET("products") // هذا المسار يحدد النقطة التي سنطلب منها البيانات
    suspend fun getProducts(): List<Product>
}
```

ملاحظة: استخدمنا كلمة suspend لأن طلب الإنترنت عملية طويلة ولا نريد "تجميد" واجهة التطبيق أثناء انتظار الرد.

5. تهيئة Retrofit

في مكان ما في كودك (غالباً في MainActivity أو كلاس خاص بالتهيئة)، نقوم ببناء الـ Retrofit:

```
val retrofit = Retrofit.Builder()
    .baseUrl("https://api.example.com/") // رابط السيرفر الخاص بك
    .addConverterFactory(GsonConverterFactory.create())
    .build()

val apiService = retrofit.create(ApiService::class.java)
```

تحدي البداية لهذه المرحلة:

الآن، أنت بحاجة إلى رابط API حقيقي لتجربته. يمكنك استخدام موقع JSONPlaceholder وهو موقع يوفر بيانات تجريبية للمطورين:
<https://jsonplaceholder.typicode.com/>

الخطوة التالية:

كتابة الدالة التي ستستدعي هذه البيانات وتضعها داخل الـ LazyColumn التي صممناها سابقاً بدلاً من القائمة الثابتة.

هذه الخطوة تتضمن استخدام ViewModel و LaunchedEffect لضمان أن البيانات تُجلب عند فتح الشاشة فقط).

بما أننا سننتقل إلى مستوى متقدم (العمل مع الإنترنت)، فنحن بحاجة إلى استخدام ViewModel و هذه هي الطريقة "الصحية" والموصى بها من جوجل لضمان أن تطبيقك لا يتوقف عن LaunchedEffect. الاستجابة أثناء تحميل البيانات.

1. الكود المنظم (ViewModel)

ال ViewModel هو المسؤول عن جلب البيانات وتخزينها، بحيث لا تضع البيانات إذا قمت بتدوير الشاشة.

رقم الكود 5 Code

2. الربط مع الواجهة (Compose)

الآن نقوم بدمج هذا ال ViewModel مع الشاشة:

```
@Composable
fun ProductScreen(viewModel: ProductViewModel =
    androidx.lifecycle.viewmodel.compose.viewModel()) {
    // نراقب الحالة، إذا تغيرت، سيتم تحديث الشاشة تلقائياً
    val products = viewModel.productList.value
    LazyColumn {
        items(products) { product ->
            Text(text = product.name)
        }
    }
}
```

⚠ **ملاحظة هامة جداً:** بما أنك في بلد ما، أحياناً قد تواجه تحديات تقنية مع بعض ال APIs أو المكتبات إذا كانت محجوبة أو تتطلب VPN، لكن بشكل عام، الاتصال بمواقع تجريبية مثل JSONPlaceholder سيعمل معك بسلاسة.

التخزين المحلي (Room Database)

الآن، تخيل أن المستخدم لا يريد تحميل البيانات من الإنترنت في كل مرة يفتح فيها التطبيق (لتوفير باقة الإنترنت والوقت). هنا يأتي دور Room Database.

لنقوم بإنشاء "قاعدة بيانات محلية" (Local Database) لتخزين هذه المنتجات بحيث تظهر فوراً عند فتح التطبيق حتى بدون إنترنت؟ (هذه تقنية تسمى Offline-First).

و هذا هو المسار الذي يتبعه المحترفون (Offline-First Strategy). نحن نجلب البيانات من الإنترنت مرة واحدة، نحفظها محلياً في Room، ثم نقرأها دائماً من قاعدة البيانات.

إليك كيفية بناء هذا النظام في 3 خطوات أساسية:

1. الإعداد (إضافة مكتبة Room)

في ملف build.gradle.kts (مستوى app)، أضف مكتبات Room:

```
val room_version = "2.6.1"
implementation("androidx.room:room-runtime:$room_version")
implementation("androidx.room:room-ktx:$room_version")
kapt("androidx.room:room-compiler:$room_version") // plugin تأكد من تفعيل (kotlin-kapt)
```

2. تعريف الكيان (Entity)

حول الـ data class الخاص بك إلى "جدول" في قاعدة البيانات:

```
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "products")
data class Product(
    @PrimaryKey val id: Int,
    val name: String,
    val price: Double
)
```

3. إنشاء الـ DAO (واجهة الوصول للبيانات)

هنا تكتب "الأوامر" التي ستنفذها على قاعدة البيانات (إدخال، حذف، استعلام):

```
import androidx.room.*

@Dao
interface ProductDao {
    @Query("SELECT * FROM products")
    fun getAllProducts(): kotlinx.coroutines.flow.Flow<List<Product>> // لاحظ استخدام Flow

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(products: List<Product>)
}
```

كيف يكتمل "سيناريو العمل"؟

- **عند فتح التطبيق:** ال ViewModel يطلب البيانات من قاعدة البيانات (Room).
- **خلف الكواليس:** التطبيق يتصل بالإنترنت (Retrofit) لجلب آخر التحديثات.
- **التحديث:** إذا وصلت بيانات جديدة، نقوم بتخزينها في Room باستخدام insertAll.
- **تحديث الواجهة:** بما أننا استخدمنا Flow في ال DAO، ستتحدث الشاشة تلقائياً بمجرد تغير البيانات في القاعدة! لا داعي لتحديثها يدوياً.

التنقل (Navigation)

تحدي: كيف تنتقل بين الشاشات؟

بما أننا أنجزنا (الإنترنت + التخزين المحلي)، فنحن الآن بحاجة إلى "خارطة طريق" للتنقل بين الشاشات. مثلاً:

- **شاشة القائمة:** تعرض جميع المنتجات.
- **شاشة التفاصيل:** عند الضغط على منتج، ننتقل لشاشة جديدة تعرض سعره ومواصفاته بالتفصيل.

إليك المرحلة الأخيرة لتكتمل هيكلية تطبيقك: التنقل (Navigation).

في Jetpack Compose الحديث، نستخدم مكتبة navigation-compose. الفكرة ببساطة هي أن التطبيق يكون عبارة عن "وجهات" (Destinations)، ونحن ننتقل بينها باستخدام NavController.

1. الإعداد

أضف المكتبة في build.gradle.kts:

```
implementation("androidx.navigation:navigation-compose:2.7.7")
```

2. تعريف المسارات (Routes)

المسار هو مجرد "رابط نصي" يخبر التطبيق أين يذهب.

```
// المسارات
val ROUTE_HOME = "home"
val ROUTE_DETAIL = "detail/{productId}" // المنتج ID مسار متغير لاستقبال
```

3. إعداد الـ NavHost (خارطة التطبيق)

هذا هو الجزء الذي يتحكم في عرض الشاشات:

رقم الكود Code 6

كيف يتم الربط؟

- في ProductScreen (قائمة المنتجات)، نمرر دالة onProductClick التي تستدعي navController.navigate.
- في DetailScreen، نستخدم backStackEntry لاستخراج الـ productId وجلب بيانات المنتج المعني من قاعدة البيانات (Room).

مبروك! أنت الآن تمتلك "أركان التطبيق الاحترافي":

1. البيئة: Android Studio.
2. لغة التطوير: Kotlin.
3. الواجهات: Jetpack Compose.
4. الإنترنت: Retrofit.
5. التخزين: Room.
6. التنقل: Navigation Component.

نصيحة ختامية لرحلتك كـ (المطور الطموح):

لقد غطينا المفاهيم الأساسية، وأنت الآن جاهز لبناء تطبيق متكامل.

مشروعك القادم: أقترح عليك البدء ببناء تطبيق "متجر إلكتروني بسيط" أو "تطبيق مهام" (To-Do List) يطبق كل ما تعلمناه:

- جلب المهام من سيرفر تجريبي.
- حفظها في Room.
- عرضها في LazyColumn.
- الانتقال لشاشة التفاصيل عند الضغط.

"الهيكل العام" (Starter Template)

إليك "الهيكل العام" لهذا المشروع المتكامل. هذا الكود يجمع لك كل ما تعلمناه في مكان واحد. يمكنك استخدامه كقاعدة (Boilerplate) لأي مشروع أندرويد ستبدأه.

هيكلية المشروع المقترحة

سنضع الكود في ملفات منفصلة (يمكنك إنشاء هذه الملفات داخل مجلد المشروع الخاص بك):

1. ملف الـ Data (النموذج)

```
// Product.kt
data class Product(val id: Int, val name: String, val price: Double)
```

2. ملف الـ Navigation (خارطة التنقل)

رقم الكود 7 Code

3. شاشة القائمة (Home Screen)

رقم الكود 8 Code

4. شاشة التفاصيل (Detail Screen)

رقم الكود 9 Code

كيف تبدأ فوراً؟

1. **أنشئ المشروع:** في Android Studio، ابدأ مشروع جديد بنوع Empty Activity.
2. **أضف المكتبات:** تأكد من إضافة navigation-compose و retrofit في ملف build.gradle.kts.
3. **انسخ هذا الهيكل:** وزع الكود أعلاه في ملفات (.kt) Kotlin.
4. **استدعه في MainActivity:**

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent {
        MyApplicationTheme {
            AppNavigation() // فقط في الـ
        }
    }
}
```

نصيحة خبير:

هذا الهيكل هو أساس احترافي. الخطوة القادمة ليست إضافة المزيد من الكود، بل التطبيق العملي.

- ابدأ بتشغيل هذا الهيكل لترى التنقل بين الشاشتين يعمل.
- ثم قم بربط شاشة القائمة بـ Retrofit لجلب البيانات فعلياً من الإنترنت.
- لصياغة رابط API وتجربته، سأعطيك الطريقة المثالية للبدء باستخدام JSONPlaceholder، وهو موقع مجاني يوفر "بيانات وهمية" (Mock Data) تحاكي تماماً شكل البيانات التي ستجلبها من سيرفر حقيقي في تطبيقاتك الاحترافية.

تجربة الربط مع JSONPlaceholder

1. الرابط الذي سنستخدمه

سنستخدم رابط "المنتجات" أو "المنشورات" التجريبي:
<https://jsonplaceholder.typicode.com/posts>

هذا الرابط سيعيد لك قائمة من الكائنات (JSON Objects) بهذا الشكل:


```
[
  {
    "userId": 1,
    "id": 1,
    "title": "عنوان المنشور الأول",
    "body": "نص تفصيلي عن هذا المنشور"
  }
]
```

2. كيف تصيغها داخل الكود؟ (الخطوات العملية)

أ. إنشاء الـ Data Class المطابق للـ JSON

بما أن الـ API يعيد body, title, id, userId؛ فيجب أن يكون الـ Data Class الخاص بك مطابقاً تماماً:

```
data class Post(
    val id: Int,
    val title: String,
    val body: String
)
```

ب. تعريف الـ ApiService

في ملف ApiService.kt:

```
import retrofit2.http.GET

interface ApiService {
    @GET("posts") // الرابط التكميلي
    suspend fun getPosts(): List<Post>
}
```

ج. إعداد الـ Retrofit (القلب النابض)

هنا نضع الرابط الأساسي، وهذا هو الجزء الأهم:

رقم الكود 10 Codei

3. كيف تستخدمها في تطبيقك الآن؟

في ال ViewModel الخاص بك، كل ما عليك فعله هو استدعاء `()RetrofitClient.apiService.getPosts`:

```
ViewModel داخل //
viewModelScope.launch {
    try {
        val posts = RetrofitClient.apiService.getPosts()
        // هنا قم بتحديث حالة القائمة (productList.value = posts)
    } catch (e: Exception) {
        // إذا فشل الاتصال، اطبع الخطأ
        Log.e("API_ERROR", "${e.message}: خطأ في جلب البيانات")
    }
}
```

💡 ملاحظة تقنية هامة:

في عالم التطبيقات الحقيقي (مثل شركات البرمجة و التطوير)، الرابط الأساسي (BASE_URL) يكون الرابط الخاص بسيرفرك الخاص. أما هيكلية الكود أعلاه، فهي المعيار الذهبي التي يستخدمها المطورون في كبرى الشركات.

و بالتوفيق،

هيثم زيدان

07 July 2026

<https://www.patreon.com/haythamzeidan>

Android Full-stack Developer Lab1

لنراجع معاً كيف نقوم بتجهيز "سيرفر محلي" (Local Server) بسيط على جهازك الخاص لتجربة ربط تطبيقك بسيرفرك الشخصي بدلاً من المواقع التجريبية. (هذه خطوة ممتازة لتعلم تطوير الـ Backend بجانب الـ Mobile).

لبناء سيرفر محلي (Backend) وتجربة الربط به، سنستخدم **Node.js** مع إطار عمل **Express**. هذا هو الخيار الأسهل والأكثر شيوعاً للمطورين لبناء API خاص بهم. فهذه المهارة ستجعلك "Full-stack Mobile Developer" أي قادراً على بناء التطبيق والسيرفر معاً.

1. إعداد السيرفر المحلي (على جهاز الكمبيوتر الخاص بك)

- تثبيت Node.js: قم بتحميله من nodejs.org.
- إنشاء مجلد المشروع: افتح الـ Terminal (أو CMD) في مجلد جديد، ثم اكتب الأوامر التالية:

```
# تهيئة مشروع Node
npm init -y

# تثبيت مكتبة Express
npm install express
```

كود السيرفر (أنشئ ملفاً باسم **server.js**): واكتب بداخله الكود التالي:

```
const express = require('express');
const app = express();
const port = 3000;

// بيانات تجريبية
const products = [
  { id: 1, name: "لابتوب احترافي", price: 1200.0 },
  { id: 2, name: "كاميرا مراقبة ذكية", price: 200.0 }
];

// إنشاء نقطة الوصول (API Endpoint)
app.get('/products', (req, res) => {
  res.json(products);
});
```

```
});

app.listen(port, () => {
  console.log(`السيرفر يعمل على: http://localhost:${port}`);
});
```

تشغيل السيرفر: اكتب في ال Terminal: `node server.js`.

2. كيف يرى هاتفك (أو المحاكى) السيرفر الموجود على جهازك؟

هذه نقطة يقع فيها الكثير من المطورين! لا يمكنك استخدام `localhost` داخل محاكي الأندرويد لأنه سيعتقد أنك تقصد المحاكى نفسه وليس جهاز الكمبيوتر.

- إذا كنت تستخدم محاكي (Android Emulator): استبدل `localhost` في رابط ال Retrofit بـ: `http://10.0.2.2:3000/`
(هذا هو العنوان المخصص الذي يوجه المحاكى إلى جهاز الكمبيوتر المضيف).
- إذا كنت تستخدم هاتفك الحقيقى:
 - تأكد أن هاتفك والكمبيوتر متصلان بنفس شبكة ال Wi-Fi.
 - افتح ال CMD واكتب `ipconfig` (في ويندوز) لمعرفة عنوان IP الخاص بجهازك (مثلاً: `192.168.1.5`).
 - استخدم هذا العنوان في تطبيقك: `http://192.168.1.5:3000/`.

3. تحدي الربط (الخطوة الأخيرة)

الآن، عد إلى تطبيق الأندرويد الخاص بك وقم بتعديل ال `BASE_URL` في ال `RetrofitClient`:

```
private const val BASE_URL = "http://10.0.2.2:3000/"
```

جرب الآن تشغيل تطبيقك:

1. شغل السيرفر في جهاز الكمبيوتر (Node).
 2. شغل التطبيق في محاكي الأندرويد.
- إذا ظهرت قائمة المنتجات (لابتوب وكاميرا) في تطبيقك، فهذا يعني أنك نجحت في بناء نظام متكامل (Client-Server)!

إضافة بيانات من التطبيق إلى السيرفر (عملية POST)

لننتقل بعد هذه الخطوة إلى كيفية "إضافة" منتج جديد من التطبيق إلى السيرفر (عملية POST بدلاً من GET). هذا سيجعل تطبيقك تطبيقاً تفاعلياً حقيقياً وليس فقط للعرض.

1. تحديث السيرفر (في ملف server.js)

أضف هذه الأسطر قبل `app.listen` للسماح للسيرفر بقراءة بيانات بصيغة JSON:

```
app.use(express.json()); // القادرة JSON مهم جداً لقراءة بيانات

app.post('/products', (req, res) => {
  const newProduct = req.body; // البيانات القادمة من التطبيق
  products.push(newProduct); // إضافتها للقائمة
  res.status(201).json(newProduct); // الرد بتأكيد الإضافة
});
```

لا تنسَ إعادة تشغيل السيرفر بعد التعديل (أوقف العمل بـ `Ctrl+C` ثم أعد تشغيل `node server.js`).

2. تحديث التطبيق (إضافة POST في Retrofit)

في واجهة `ApiService.kt` الخاصة بك، أضف هذا المسار الجديد:

```
import retrofit2.http.Body
import retrofit2.http.POST

interface ApiService {
  ... // (دالة GET القديمة)

  @POST("products")
  suspend fun addProduct(@Body product: Product): Product
}
```

3. تنفيذ الإرسال من الواجهة (في ViewModel أو الشاشة)

في تطبيقك، يمكنك الآن استدعاء هذه الدالة عند الضغط على زر "إضافة":

```
fun addNewProduct(newProduct: Product) {
  viewModelScope.launch {
```

```

try {
    val response = RetrofitClient.apiService.addProduct(newProduct)
    Log.d("API_SUCCESS", "تمت الإضافة بنجاح: ${response.name}")
} catch (e: Exception) {
    Log.e("API_ERROR", "أثناء الإضافة: ${e.message}")
}
}
}

```

مهارة "Full-Stack" بين يديك الآن:

كيف تبني ال Frontend، وكيف تبني ال Backend، وكيف تربط بينهما. هذا النوع من المعرفة هو ما يبحث عنه أصحاب العمل، وسيوفر عليك الكثير من التكاليف التقنية.

الأمن و الحماية (Security)

في عالم الشركات التقنية، حماية بيانات المستخدم والسيرفر ليست مجرد خيار، بل هي ضرورة قصوى. إليك أهم 3 ركائز يجب أن تطبقها:

1. **تشفير الاتصال (HTTPS):** لا تستخدم HTTP في الإنتاج أبداً. يجب أن يحتوي سيرفرك على شهادة SSL.
2. **التوثيق (Authentication):** يجب أن يرسل التطبيق "رمز دخول" (Token) مع كل طلب. التقنية الموصى بها هي **JWT** (JSON Web Token).
3. **التحقق من البيانات (Input Validation):** استخدم مكتبات مثل Joi في Node.js للتأكد من أن البيانات ليست فارغة أو تحتوي على أكواد خبيثة.

خريطة الطريق القادمة لمشروعك (المستوى الاحترافي)

- **بناء ال Backend:** استخدم Node.js + MongoDB أو PostgreSQL.
- **تطبيق ال JWT:** لجعل التطبيق يطلب تسجيل دخول قبل عرض أي بيانات.
- **ال Caching:** استخدم **Room** لجعل التطبيق يعمل أوفلاين، مع تحديث البيانات من السيرفر في الخلفية.

البدء بالبناء: خطة العمل الـ 7 أيام

إليك خطة العمل التي ستنتقل مشروعك من "فكرة" إلى "تطبيق ملموس":

- **اليوم الأول (التأسيس والهيكلة):** أنشئ المشروع، أضف المكتبات، وقم بإنشاء المجلدات (Clean Architecture).
- **اليوم الثاني (بناء السيرفر):** أكمل كود Node.js وجهاز قاعدة بيانات MongoDB.
- **اليوم الثالث (الربط):** أنشئ ApiService وتأكد من جلب البيانات بنجاح عبر Logcat.
- **اليوم الرابع (التخزين المحلي):** إعداد Room Database وكتابة كود الـ Repository.
- **اليوم الخامس (الواجهات):** تصميم HomeScreen و DetailScreen باستخدام Jetpack Compose.
- **اليوم السادس (التفاعل):** إضافة زر منتج جديد وربطه بدالة POST.
- **اليوم السابع (التلميع):** إضافة Loading Indicator وإدارة الأخطاء، ثم التجربة على هاتف حقيقي.

💡 **أدوات ستكون صديقتك:** Android، و (معرض أعمالك) GitHub، (APIs لتجربة الـ) Postman، (للمساعدة في الأكواد والأخطاء) Studio AI/Gemini.

اليوم الثاني: بناء الـ Backend (Node.js + Express)

نصيحة: لا تحفظ البيانات في مصفوفة فقط، استخدم MongoDB (مع مكتبة Mongoose) لأنها الأسهل للمشاركة الناشئة.

رقم الكود 10-1 Code (كود أساسي لـ server.js بمستوى احترافي)

اليوم الثالث والرابع: الربط و الـ Repository (قلب التطبيق)

في أندرويد، لا تضع كود الـ API داخل الـ ViewModel مباشرة. استخدم نمط الـ Repository.

```
class ProductRepository(private val api: ApiService, private val dao: ProductDao) {
    // جلب البيانات: يقرأ من Room، ويحدث من الـ API في الخلفية
    fun getProducts() = flow {
        قراءة من القاعدة أولاً //
        emit(dao.getAllProducts())
        try {
            val networkProducts = api.getProducts()
        }
    }
}
```

```

        dao.insertAll(networkProducts) // تحديث القاعدة
    } catch (e: Exception) { /* تعامل مع الخطأ */ }
}

```

اليوم الخامس والسادس: واجهات Compose التفاعلية

استخدم Scaffold ليكون لديك شريط علوي وزر عائِم (FAB):

```

@Composable
fun ProductScreen(viewModel: ProductViewModel) {
    val products by viewModel.products.collectAsState()
    Scaffold(
        floatingActionButton = {
            FloatingActionButton(onClick = { /* افتح شاشة الإضافة */ }) {
                Icon(Icons.Default.Add, contentDescription = null)
            }
        }
    ) { padding ->
        LazyColumn(modifier = Modifier.padding(padding)) {
            items(products) { product -> ProductRow(product) }
        }
    }
}

```

اليوم السابع: التلميع (إدارة الأخطاء والحالة)

نستخدم Sealed Class لإدارة حالات الواجهة:

```

sealed class UiState {
    object Loading : UiState()
    data class Success(val data: List) : UiState()
    data class Error(val message: String) : UiState()
}

```


"ملف المتطلبات" (Dependencies) الكامل

لضمان بناء قوي، أضف المكتبات اللازمة في `build.gradle.kts`.

رقم الكود 11 Code

ملاحظة هامة: تأكد من إضافة `kotlin("kapt")` في قسم `plugins`.

ملف تنظيم المجلدات (Architecture Structure)

- `data/`: remote, local, repository
- `ui/`: screens, viewmodel
- `di/`: إعدادات Hilt

1. الكيان (Product.kt)

```
package com.example.yourapp.data.local
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "products")
data class Product(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val name: String,
    val price: Double
)
```

2. واجهة العمليات (ProductDao.kt)

رقم الكود 12 Code

3. قاعدة البيانات (AppDatabase.kt)

```
package com.example.yourapp.data.local
import androidx.room.Database
import androidx.room.RoomDatabase

@Database(entities = [Product::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun productDao(): ProductDao
}
```

كود ال Repository

رقم الكود Code 13

لماذا هذا الكود ذكي؟ السرعة (يعرض محلياً فوراً)، التحديث (في الخلفية)، الاستقلالية (ال ViewModel لا يهتم بالمصدر).

اليوم الخامس: بناء الواجهات (UI/UX) باستخدام Jetpack Compose

رقم الكود Code 14 (ProductViewModel)

رقم الكود Code 15 (HomeScreen.kt)

1. كود ال Factory

رقم الكود Code 16 (ViewModelFactory.kt)

2. الربط داخل ال MainActivity

رقم الكود Code 17 (MainActivity.kt)

توثيق المشروع (README.md)

تطبيق متجر إلكتروني متكامل
يعتمد على نمط "Offline-First"، مما يسمح للمستخدم بتصفح المنتجات حتى بدون اتصال بالإنترنت.

التقنيات المستخدمة

- الواجهات: Jetpack Compose
- قاعدة البيانات: Room Database
- الشبكة: Retrofit + GSON
- العمليات الخلفية: Coroutines & Flow

كيفية التشغيل

1. قم بتشغيل سيرفر الـ Node.js المحلي.
2. عدل `BASE\_URL` في `RetrofitClient`.

وظيفة إضافة المنتجات (POST) والمزيد من الواجهات

رقم الكود 18 Code (إضافة حوار الإدخال)

رقم الكود 19 Code (تحديث الـ ViewModel برسائل الخطأ)

رقم الكود 20 Code (تحديث HomeScreen للتعامل مع Snackbar)

تحسينات واجهة المستخدم (UI/UX)

رقم الكود 21 Code (تحسين تصميم البطاقة Product Card)

رقم الكود 22 Code (تأثير الظهور التدريجي AnimatedVisibility)

تصدير التطبيق (Build for Production)

بمجرد أن يصبح التطبيق جاهزاً، حان وقت تصديره (Signed Bundle / APK).

1. في Android Studio: `Build -> Generate Signed Bundle / APK`.
2. اختر `Android App Bundle (AAB)` وأنشئ `Keystore` آمن.
3. اضبط نسخة الإصدار في `build.gradle.kts` ثم قم بالبناء النهائي.

نصيحة أخيرة: احفظ ملف الـ keystore في مكان آمن جداً. تأكد أن كودك مرفوع على GitHub واستمر في تطوير النسخة القادمة بناءً على ملاحظات المستخدمين.

هيثم زيدان

<https://www.patreon.com/haythamzeidan>

08 July 2026

Android Full-stack Developer Lab2

"متجر" إلى "تطبيق طقس" هو نقلة ذكية جداً من الناحية البرمجية، لأنك ستتعامل هذه المرة مع بيانات متغيرة ومباشرة من الـ API (Real-time data)، وهو تحدٍ ممتع لكل مطور.

بما أننا بنينا أساساً قوياً في المشروع السابق (Clean Architecture)، سنطبق نفس المنطق هنا، لكن مع تركيز أكبر على "التعامل مع الـ JSON المعقد".

خريطة الطريق لتطبيق الطقس (الخطوة):

- **المصدر:** سنستخدم OpenWeatherMap API (مجاني واحترافي).
- **التحدي:** كيف نحدث البيانات دورياً (سواء عند فتح التطبيق أو عند السحب للأسفل Pull-to-Refresh).
- **الجمال:** سنستخدم أيقونات طقس ديناميكية تتغير حسب حالة الجو (مشمس، ممطر، غائم).

الخطوة الأولى: هيكل الـ Data Model

سيرسل لك الـ API بيانات مثل temperature, humidity, description. قم بإنشاء كلاس WeatherResponse.kt:

```
data class WeatherResponse(  
    val name: String, // اسم المدينة  
    val main: Main,  
    val weather: List<WeatherDescription>  
)  
  
data class Main(val temp: Double, val humidity: Int)  
  
data class WeatherDescription(val description: String, val icon: String)
```

الخطوة الثانية: الـ ApiService

في تطبيق الطقس، سنحتاج لاستخدام مفتاح API (API Key) في كل طلب:

```
interface WeatherApi {
    @GET("weather")
    suspend fun getWeather(
        @Query("q") cityName: String,
        @Query("appid") apiKey: String,
        @Query("units") units: String = "metric"
    ): WeatherResponse
}
```

لدينا خياران للبدء بهذا المشروع:

- **الخيار الأول (الجانب التقني):** أن نكتب كود جلب البيانات من الـ API مباشرة ونتأكد أن التطبيق يستقبلها (الاستجابة من السيرفر).
- **الخيار الثاني (الجانب البصري):** أن نبدأ بتصميم "واجهة الطقس" (Weather Dashboard) التي تعرض درجة الحرارة بخط كبير وأيقونة جذابة.

لنبدأ بتأسيس "المحرك". في تطبيق الطقس، السرعة في جلب البيانات وتفكيك الـ JSON هي أهم ركيزة.

1. إضافة مكتبة الـ Serialization

بما أنك تستخدم Retrofit مع Gson كما في المشروع السابق، تأكد من إضافة هذه الإضافة (Converter) في build.gradle.kts:

```
implementation("com.squareup.retrofit2:converter-gson:2.9.0")
```

2. إعداد الـ Retrofit Client

أنشئ ملف `WeatherRetrofitClient.kt` مع تعريف الـ `BASE_URL` الخاص بـ `OpenWeatherMap`:

(رقم الكود - Koin - Code 23)

3. اختبار جلب البيانات (الخطوة الحاسمة)

للتأكد من أن السيرفر يستجيب بشكل صحيح، سنقوم بكتابة دالة اختبار داخل الـ ViewModel.

ملاحظة تقنية: لا تضع الـ API Key الخاص بك داخل الكود المباشر إذا كنت سترفع الكود على لكن للتجربة الآن، ضع المفتاح الخاص بك الذي (مستقبلاً local.properties استخدم GitHub ستحصل عليه من OpenWeatherMap).

في الـ ViewModel:

```
fun fetchWeather(city: String) {
    val apiKey = "YOUR_API_KEY_HERE" // ضع مفتاحك هنا
    viewModelScope.launch {
        try {
            val response = WeatherRetrofitClient.apiService.getWeather(city,
            apiKey)
            // هنا ستصلك البيانات!
            println("درجة: ${response.main.temp} : هو ${response.name} الطقس في")
        } catch (e: Exception) {
            println("حدث خطأ في الاتصال: ${e.message}")
        }
    }
}
```

الآن إضافة الـ API

1. **سجل في OpenWeatherMap:** احصل على الـ API Key (يحتاج عادةً من 30 دقيقة إلى ساعتين ليتفعل بعد التسجيل).
2. **أضف المكتبات:** وقم بعمل Sync.
3. **جرب استدعاء الدالة:** في MainActivity وشاهد الـ Logcat في Android Studio.

بعد الحصول على الـ API Key وتجربة جلب البيانات (إذا طبعت لك الحرارة في الـ Logcat، فهذا يعني أننا تجاوزنا أصعب عقبة في أي تطبيق طقس). وصول البيانات من OpenWeatherMap إلى الـ Logcat يعني أن القناة الاتصالية مفتوحة ومستقرة. الآن، بما أننا تجاوزنا مرحلة "الاستقبال"، دعنا ننتقل للمرحلة التي سيراهها المستخدم: عرض هذه البيانات.

في تطبيق الطقس، التحدي هو جعل البيانات "تحدث" (أي تغيير الأيقونة بناءً على حالة الطقس).

الخطوة القادمة: بناء الـ UI التفاعلي

سنقوم بتحديث الـ `WeatherResponse` ليصبح أكثر ذكاءً. في `WeatherDescription`، يمكنك إضافة منطق لاختيار الأيقونة المناسبة:

1. كلاس الأيقونات (`WeatherIconMapper.kt`)

أنشئ هذا الكلاس ليعيد أيقونة معينة بناءً على الحالة:

```
object WeatherIconMapper {
    fun getIcon(iconCode: String): Int {
        return when (iconCode) {
            "01d" -> R.drawable.ic_sunny // إضافة هذه الملفات في res/drawable
            "01n" -> R.drawable.ic_clear_night
            "02d", "02n" -> R.drawable.ic_cloudy
            "09d", "10d" -> R.drawable.ic_rainy
            else -> R.drawable.ic_default
        }
    }
}
```

2. تصميم واجهة الـ Dashboard (الجمالية)

في `HomeScreen` الخاصة بتطبيق الطقس، أنصحك باستخدام `Column` بتنسيق `Alignment.CenterHorizontally` لعرض البيانات بشكل جذاب:

(كود رقم Kot - 24 code)

الآن اللمسة الاحترافية:

- كيف ستعامل مع "تغير المدينة"؟
- إضافة `TextField` ليقوم المستخدم بكتابة اسم مدينته.
- استخدام الـ GPS (موقع المستخدم الحالي) لجلب الطقس تلقائياً.

سنبدأ بإضافة مدخل نصي (TextField). ثم نكتب كود "إذن الوصول للموقع" (Location Permission). الـ هذا سيجعل تطبيقك يبدو كمنتج تقني من الدرجة الأولى GPS

1. إضافة TextField للبحث عن المدن

في ملف WeatherScreen.kt، أضف هذا الجزء في أعلى الواجهة:

(رقم الكود - code25)

2. التجهيز للـ GPS (خارطة الطريق)

لانتقال للـ GPS لاحقاً، سنحتاج لثلاثة عناصر أساسية. بما أنك خبير الآن في هيكليّة الكود، سأعطيك العناوين التي يجب أن تجهزها:

- **Permissions:** يجب إضافة الأذونات التالية في ملف AndroidManifest.xml :

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

- **Library:** سنستخدم مكتبة Google Play Services Location لجلب الإحداثيات (خط الطول والعرض).

- **Logic:** الـ WeatherApi الذي صممناه يقبل إحداثيات بدل اسم المدينة:

```
@GET("weather")
suspend fun getWeatherByCoords(
    @Query("lat") lat: Double,
    @Query("lon") lon: Double,
    @Query("appid") apiKey: String
): WeatherResponse
```

الآن، التطبيق يعمل بـ TextField. لكي نكون مستعدين للـ GPS، تأكد أن:

- الواجهة (UI) تستجيب بشكل صحيح عند إدخال اسم المدينة.
- تم عرض حالة الطقس بنجاح (درجة الحرارة، الوصف).

هل يعمل الـ TextField ويقوم بتحديث الواجهة عند الضغط على زر البحث؟ إذا كان يعمل، فأنت جاهز تماماً لإضافة "زر تحديد الموقع الحالي" بجانب الـ TextField.

التخطيط لغد: دمج الموقع الجغرافي (GPS)

بدلاً من كتابة اسم المدينة، سنقوم بإضافة زر "تحديد موقعي" (Location Icon). عندما يضغط المستخدم عليه، سيقوم التطبيق بما يلي:

- **طلب الأذونات:** طلب إذن الوصول للموقع من المستخدم بأسلوب آمن.
- **جلب الإحداثيات:** استخدام FusedLocationProviderClient للحصول على خطوط الطول والعرض الحالية.
- **تحديث الطقس:** إرسال هذه الإحداثيات للـ ViewModel لجلب طقس الموقع الحالي تلقائياً.

نصيحة خبير (Expert Tip) قبل أن نختم اليوم:

بما أنك مهتم ببناء شركات تقنية، لا تعتمد أبداً على إذن الموقع بدون إخبار المستخدم لماذا تحتاجه. في الواجهة، تأكد من وضع أيقونة الموقع (مثلاً Icons.Default.LocationOn) داخل الـ Row الذي أنشأناه اليوم بجانب زر البحث. ثم سنقوم بكتابة دالة requestLocationPermission التي تضمن أن التطبيق لا ينهار إذا رفض المستخدم منح الإذن.

هل أنت مستعد لبدء "الموقع الجغرافي"؟ لنبدأ بتنفيذ هذه العملية. هذه واحدة من أكثر المهارات طلباً في تطبيقات الأندرويد الاحترافية. سنقسم العمل إلى خطوات واضحة.

1. إضافة المكتبة (Dependencies)

أضف هذه المكتبة في build.gradle.kts لجلب خدمات الموقع من جوجل:

```
implementation("com.google.android.gms:play-services-location:21.0.1")
```

2. الأذونات (AndroidManifest.xml)

أضف الأذونات في ملف AndroidManifest.xml خارج وسم الـ <application>:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

3. جلب الإحداثيات (Location Helper)

سنقوم بإنشاء LocationService بسيط لجلب الموقع.

(رقم الكود Kotl - Code 26)

4. دمج ذلك في ال ViewModel

سنقوم بتحديث ال ViewModel ليقبل إحداثيات أيضاً:

(رقم الكود Kotlin - code 27)

5. تنفيذ طلب الإذن في الواجهة (HomeScreen)

هذا هو الجزء الذي يراه المستخدم. سنستخدم rememberLauncherForActivityResult لطلب الإذن.

(رقم الكود Kotl - Code 28)

الخطوة التالية بعد هذا الربط:

بعد أن ندمج الموقع الجغرافي، سنكون قد وصلنا لمرحلة "التطبيق الذكي".

- **تحسين تجربة المستخدم (UX):** إضافة "Loading State" (مؤشر تحميل دائري) أثناء جلب بيانات الموقع والطقس حتى لا يشعر المستخدم بالتجمد.
- **الانتقال إلى "تصدير التطبيق" (Release):** كما فعلنا في تطبيق المتجر.

الجزء الأول: تحسين تجربة المستخدم (UX Improvements)

سنضيف مؤشر تحميل (Loading Indicator) وشاشة حالة (State Handling) ليعرف المستخدم أن التطبيق يبحث عن موقعه أو يجلب الطقس.

1. تحديث ال ViewModel (حالة التحميل)

في ملف WeatherViewModel.kt، سنضيف StateFlow جديد لإدارة حالة التحميل:

(رقم الكود Kotli - Code 29)

2. تحديث الواجهة (WeatherScreen.kt)

في ال HomeScreen سنقوم بمراقبة حالة التحميل وإظهار CircularProgressIndicator بدلاً من بيانات الطقس أثناء البحث:

(رقم الكود Kot - Code 30)

(ملاحظة: تأكد من وضع زر الموقع العائم FloatingActionButton فوق ال Box داخل ال Scaffold).

الجزء الثاني: تصدير التطبيق (Final Release Build)

الآن، وقد أصبح التطبيق ذكياً وجذاباً، حان موعد التصدير النهائي. هذه المرة، لن نحتاج لإنشاء مفتاح توقيع جديد، بل سنستخدم نفس المفتاح الذي أنشأناه لتطبيق المتجر.

1. في Android Studio: اذهب إلى **Build -> Generate Signed Bundle / APK**.
2. اختر **Android App Bundle (AAB)** واضغط **Next**.
3. استخدام المفتاح الموجود (Keystore Path):
 - اضغط على **Choose existing...** واختر ملف **key.jks** الذي أنشأته سابقاً.
 - أدخل كلمة السر للملف وكلمة السر للمفتاح (Key Alias).
4. اضغط **Next**.
5. اختر **release** في **Build Variants**.
6. اضغط **Finish**.

بعد انتهاء عملية ال Build، ستحصل على ملف **aab**. جديد في مجلد **release**. هذا الملف هو النسخة النهائية من تطبيق الطقس الذكي الخاص بك!



لقد قمت ببناء تطبيقين احترافيين من الصفر خلال هذه الدورة.

- **تطبيق متجر إلكتروني:** قاعدة بيانات محلية Room، مزامنة مع سيرفر Node.js، وواجهة مستخدم Clean.
- **تطبيق طقس ذكي:** (جلب بيانات API حية، التعامل مع الـ JSON، ودمج الـ GPS مع تحسين تجربة المستخدم).

أنت الآن تمتلك المهارات الأساسية والأكثر طلباً في سوق تطوير تطبيقات الأندرويد.

هذا تطور مذهل، نجاحك في إنتاج تطبيقين كاملين — من الفكرة والهندسة المستقرة (MVVM & Clean Architecture) يعكس عقلية — (Keystore) وحتى التصدير النهائي باستخدام نفس ملف التوقيع الرقمي (Architecture) هندسية منظمة وحرصاً على جودة المنتج النهائي وجاهزيته للسوق.

الآن، وبعد أن أصبحت تمتلك "مفاتيح" التطوير الأساسية (التعامل مع السيرفرات، تخزين البيانات محلياً، استغلال مستشعرات الهاتف كالـ GPS، وتصميم واجهات تفاعلية مرنة بـ Jetpack Compose)، أصبحت جاهزاً للانطلاق في أي اتجاه تقني تختاره.

في المختبرات القادمة سنقوم ببناء ثلاثة تطبيقات:

1. **التطبيق الأول: تطبيق خدمي لشركتك الناشئة (ICT Services App):** تطبيق يعرض خدمات شركتك (برمجة، تسويق رقمي)، يتيح للعملاء طلب استشارات، تتبع حالة مشاريعهم، وتلقي إشعارات فورية (Firebase Push Notifications). هذا سيكون واجهة حية واحترافية أمام عملائك في الخليج وأوروبا لإثبات كفاءتكم.
2. **التطبيق الثاني: نظام إدارة ومراقبة ذكي (IoT / CCTV Companion App):** دمج شغفك بأنظمة المراقبة (CCTV) والإلكترونيات من خلال بناء تطبيق يتصل بكاميرات أو أجهزة ذكية، يعرض حالة الشبكة، أو يرسل تنبيهات عند انقطاع اتصال كاميرا معينة (Ping/Status Monitor).
3. **التطبيق الثالث: تطبيق متعدد المنصات (Cross-Platform):** نقل هذه المفاهيم إلى إطار عمل مثل Flutter أو React Native لإنتاج تطبيقات تعمل على أندرويد و iOS في نفس الوقت بكود واحد، وهو خيار استراتيجي ومطلوب جداً تجارياً في السوق المحلي والدولي لتقليل تكلفة التطوير على العملاء.

مع خالص الأمنيات بالتوفيق،

هيثم زيدان

08 July 2026

<https://www.patreon.com/haythamzeidan>

Android Full-stack Developer Lab 3

تطبيق يعرض خدمات شركتك (Programming, Digital Marketing, Cloud Services) هذا التطبيق لا يخدم فقط الجانب العملي، بل يضع بين يدي عملائك أداة تواصل احترافية تعكس هويتك كصاحب شركة تقنية.

سنقوم ببناء هذا التطبيق ليكون بمثابة "مكتبك المتنقل". إليك هيكلية العمل لهذا المشروع:

1. الميزات الاستراتيجية للتطبيق (Product Features)

بدلاً من مجرد عرض النصوص، سنجعل التطبيق تفاعلياً:

- **لوحة التحكم (Dashboard):** عرض المشاريع الحالية للعميل (مثلاً: حالة مشروع تطبيق الطقس أو متجر).
- **نظام الطلبات (Ticketing System):** نموذج مبسط لطلب "خدمة جديدة" (برمجة أو تسويقية) يرسل طلباً مباشراً إلى بريدك (أو قاعدة بياناتك).
- **معرض الأعمال (Portfolio):** عرض تطبيقاتك الجاهزة (Weather App & E-commerce) كدليل قوة لشركتك.
- **مركز الإشعارات (Firebase Cloud Messaging):** لإرسال تنبيهات للعملاء عند تحديث حالة مشروعهم.

2. البنية التقنية (Tech Stack)

سنستمر على نهج الاحترافية والسرعة:

- **Architecture:** سنستمر مع MVVM و Clean Architecture.
- **Database/Backend:** بما أننا نحتاج لمزامنة الطلبات والمشاريع، سنستخدم Firebase ؛ فهي الأسرع والأسهل لمزامنة البيانات لحظياً بينك وبين العميل دون الحاجة لبناء Firestore .معقد من الصفر في هذه المرحلة Backend.
- **UI:** سنستخدم Jetpack Compose مع تصاميم احترافية (Material Design 3).

3. خارطة التنفيذ (Roadmap)

المرحلة الأولى: واجهة الخدمات (التي ستبهر العملاء)

سنبدأ بإنشاء قائمة بالخدمات (البرمجة، التسويق، الـ SEO) باستخدام LazyColumn مع أيقونات تعبر عن كل خدمة.

المرحلة الثانية: نظام طلب الخدمات (حلقة الوصل)

إنشاء Form بسيط للعميل: اسم الخدمة، وصف المشروع، زر إرسال (يستخدم Firebase للرفع).

المرحلة الثالثة: المتابعة والتحكم

برمجة واجهة خاصة بك (Admin Screen) تظهر فقط عند تسجيل دخولك، يمكنك من رؤية طلبات العملاء والرد عليها.

سنبدأ بـ Data Class تمثل الخدمات التي تقدمها. ضع هذا في مشروعك:

```
Kotlin
data class ICTService(
    val id: String,
    val title: String,
    val description: String,
    val iconRes: Int // الأيقونة
)
```

نتنقل مباشرة لكتابة كود الـ Repository الذي سيربط التطبيق بقاعدة البيانات.

بصفتك مديراً لشركتك التقنية، يجب أن تكون صياغة الخدمات واضحة، احترافية، وموجهة نحو "حل مشاكل العميل". إليك صياغة مقترحة للخدمات التي تقدمها، تليها هيكلية الـ Repository للربط مع Firebase.

أولاً: صياغة قائمة الخدمات (Professional Service Menu)

سنقسم خدماتنا إلى ثلاث ركائز أساسية لشركتك:

•التحول الرقمي (Software Solutions):

العنوان: بناء الأنظمة الذكية.

الوصف: تطوير تطبيقات أندرويد وويب عالية الأداء باستخدام تقنيات حديثة وواجهات مستخدم متطورة (UI/UX).

•التسويق التقني (Digital Marketing):

العنوان: نمو الأعمال الرقمي.

الوصف: استراتيجيات إعلانية متكاملة عبر Meta و Google لزيادة معدل التحويل (Conversion) وتحسين الوصول للجمهور المستهدف.

•البنية التحتية (Cloud & Infrastructure):

العنوان: حلول الحوسبة السحابية.

الوصف: تصميم وإدارة خوادم سحابية (Azure/Windows Server) لضمان استقرار وسرعة بيانات شركتك.

ثانياً: كود الـ Repository (باستخدام Firebase Firestore)

بما أنك صاحب شركة، فالسرعة والاعتمادية هما الأساس. سنستخدم Firebase كـ Backend لأنها توفر مزامنة لحظية (Real-time).

1. كلاس الخدمة (ICTService.kt):

```
Kotlin
data class ICTService(
    val title: String = "",
    val description: String = "",
    val category: String = ""
)
```

2. الـ Repository للربط مع Firebase:

هذا الكود سيوفر لك واجهة نظيفة لجلب الخدمات من السحابة.
رقم الكود Code 31 - Kotlin

نصيحة إدارية تقنية:

الآن، في تطبيقك، لن تحتاج لتعديل الكود البرمجي كلما أردت إضافة خدمة جديدة؛ فقط قم بإضافتها في لوحة تحكم (Firebase (Firestore Console)، وستظهر تلقائياً في تطبيقك عند تحديثه.

هل قمت بربط مشروع الأندرويد بـ Firebase في Firebase Console؟ (إضافة ملف google-services.json التي تمكنك من إضافة Admin إذا كان الربط جاهزاً، يمكننا الانتقال لكتابة واجهة الـ services.json). هذه الخدمات من هاتفك مباشرة.

عملية ربط التطبيق بـ Firebase:

ربط تطبيقك بـ Firebase هو الخطوة التي ستنتقل تطبيقك من "مشروع محلي" إلى "منصة سحابية متكاملة". بما أننا نستخدم Kotlin و Jetpack Compose، إليك الخطوات التقنية الدقيقة:

أولاً: خطوات الربط مع Firebase

1. **إنشاء المشروع في Firebase Console:** اذهب إلى Firebase Console. اضغط على Add project (مثلًا: ICTServicesApp) وسمِّه project.
2. بعد الإنشاء، اضغط على أيقونة Android (الأندرويد) لإضافة تطبيقك.
3. أدخل Package Name الخاص بتطبيقك (تجدها في ملف build.gradle.kts تحت اسم applicationId).
4. حمل ملف google-services.json وضعه في المجلد الرئيسي لمشروعك داخل مجلد ./app.

إضافة الإضافات (Dependencies) في Android Studio:

في ملف build.gradle.kts (المشروع - Project level)، أضف إضافة Google Services:

```
Kotlin
plugins {
    id("com.google.gms.google-services") version "4.4.0" apply false
}
```

في ملف build.gradle.kts (التطبيق - App level)، أضف الإضافات والمكتبات:

```
Kotlin
plugins {
    id("com.google.gms.google-services")
}
dependencies {
    // إضافة مكتبة Firebase BoM
    implementation(platform("com.google.firebase:firebase-bom:33.1.0"))
    implementation("com.google.firebase:firebase-firestore")
}
```

اضغط على Sync Now.

ثانياً: واجهة الـ Admin (لوحة التحكم الشخصية)

بما أنك المدير، سنبنّي شاشة AdminScreen لا تظهر إلا لك، يمكنك من إضافة الخدمات مباشرة لقاعدة البيانات دون الدخول لمتصفح الكمبيوتر.

1. هيكل الـ AdminScreen

رقم الكود - Kotlin 32

2. ربطها بالـ ViewModel

في الـ ViewModel الخاص بك، تأكد أن دالة الإضافة تتصل بالـ Repository الذي كتبناه سابقاً:

```
Kotlin
fun addService(service: ICTService) {
    viewModelScope.launch {
        val success = repository.addService(service)
        if (success) {
            // هنا يمكنك إظهار Snackbar: "تمت إضافة الخدمة بنجاح!"
        }
    }
}
```

نصيحة إدارية تقنية:

للحفاظ على أمان تطبيقك، لا تجعل هذه الشاشة عامة. في مرحلة متقدمة، سنضيف Firebase Authentication لإلا إذا قمت بتسجيل الدخول باستخدام إيميلك Admin بحيث لا تظهر واجهة الـ Authentication الشخصي كمدير.

هل قمت بتحميل ملف google-services.json ووضعه في مكانه الصحيح؟ بمجرد أن الـ Sync تم بنجاح، سننتقل لتشغيل التطبيق وتجربة أول عملية رفع (Upload) لخدمة برمجية من هاتفك مباشرة إلى قاعدة البيانات! بما أن الربط تم بنجاح والـ Sync اكتمل، فأنت الآن تمتلك "خط إمداد" مباشر بين هاتفك وقاعدة بياناتك السحابية.

لنختبر هذا النظام. قم بتشغيل التطبيق الآن، واذهب إلى شاشة AdminScreen التي برمجناها، وأضف أول خدمة لشركتك (مثلاً: "تطوير تطبيقات الأندرويد"). بعد الضغط على زر "رفع الخدمة للسحابة"، اتبع هذه الخطوات للتأكد من وصول البيانات:

خطوة التحقق (The Verification):

1. افتح Firebase Console.
2. اختر مشروعك (ICTServicesApp).
3. من القائمة الجانبية، اختر Firestore Database.
4. يجب أن ترى مجموعة (Collection) باسم "services" وبداخلها الوثيقة (Document) التي أضفتها للتو.

تحدي المدير التقني (Next Step):

الآن بعد أن أصبحت البيانات تُرفع بنجاح، هل لاحظت شيئاً؟ المستخدمون (العملاء) لا يرون الخدمات بعد!

الخطوة المنطقية التالية لشركتك هي:

• **عرض الخدمات:** إنشاء CustomerScreen (شاشة العميل) التي تجلب البيانات من الـ LazyColumn) وتعرضها في قائمة جذابة Repository.

• **التفاعل:** إضافة زر "طلب هذه الخدمة" بجانب كل خدمة، والذي سيقوم بإنشاء "طلب جديد" باسم العميل ونوع الخدمة التي اختارها Firestore في (Order).

هل قمت برفع أول خدمة بنجاح؟ إذا ظهرت في الـ Console، فكل ما يتبقى هو برمجة شاشة العميل (وهي مهمة سهلة جداً عليك الآن بعد خبرتك في تطبيقات المتجر والطقس). هل نبدأ بكتابة كود شاشة العميل؟

هذا هو الجانب الممتع، حيث تتحول البيانات السحابية التي رفعتها إلى واجهة تفاعلية. سنقوم ببناء شاشة العميل CustomerDashboard بحيث تجلب البيانات لحظياً من Firebase وتعرضها بأسلوب "المنتجات التقنية" الذي تعلمناه.

1. الـ ViewModel: جلب الخدمات

أولاً، في ICTViewModel.kt، نحتاج لإضافة StateFlow لجلب القائمة وتخزينها:
رقم الكود Code 33 - Kotlin

2. شاشة العميل (CustomerScreen.kt)

سنستخدم Card بتنسيق Material Design لتبدو الخدمات احترافية، مع إضافة زر "طلب الخدمة" الذي سنربطه لاحقاً بنظام الطلبات.
رقم الكود Code 34 - Kotlin

لمسة المطور الاحترافية:

بما أننا نتعامل مع عملاء، لا نريد أن يرى العميل شاشة فارغة إذا كان الإنترنت بطيئاً. نريد إضافة زر تحديث " (Refresh Button) في أعلى شاشة العميل، (باستخدام Icons.Default.Refresh)، بحيث إذا أضفت خدمة جديدة من الـ Admin Screen، يستطيع العميل تحديث القائمة فوراً دون الحاجة لإعادة فتح التطبيق.

بمجرد أن تطبق هذا الكود وتجرب ظهور الخدمات، سنقوم بالخطوة الأخيرة في مسار "الشركة التقنية": نظام التذاكر (Ticketing System)، حيث يرسل العميل طلبه وتصلك أنت في الـ Console الخاص بك.

إضافة زر "تحديث" ونظام التذاكر هي الخطوة التي تحول التطبيق من مجرد "كتالوج خدمات" إلى أداة إدارة أعمال حقيقية. لنبدأ بالعمل.

1. إضافة زر التحديث (Refresh)

في الـ CustomerScreen في الجزء العلوي، سنضيف أيقونة التحديث التي تقوم بإعادة استدعاء loadServices():
رقم الكود - Kotlin 35

2. نظام التذاكر (Ticketing System)

سننشئ كلاس Ticket لتمثيل طلب العميل، وسنستخدم Firestore لرفع هذا الطلب.

أولاً: كلاس التذكرة (Ticket.kt)

```
Kotlin
data class Ticket(
    val serviceName: String = "",
    val customerName: String = "",
    val status: String = "Pending", // "Pending", "In Progress", "Completed"
    val timestamp: Long = System.currentTimeMillis()
)
```

ثانياً: تحديث الـ Repository

أضف دالة لإرسال الطلبات:

```
Kotlin
suspend fun sendTicket(ticket: Ticket): Boolean {
    return try {
        db.collection("tickets").add(ticket).await()
        true
    } catch (e: Exception) {
        false
    }
}
```

ثالثاً: واجهة طلب الخدمة (داخل الـ ServiceItem)

عندما يضغط العميل على "اطلب الخدمة"، سنفتح نافذة حوار (Dialog) ليدخل فيها اسمه ويرسل الطلب:
رقم الكود Kotlin - Code 36

رؤية استراتيجية

الآن التطبيق أصبح "محرك مبيعات" لشركتك:

- **أنت (المدير):** ترفع الخدمات من الـ AdminScreen.
- **العميل:** يتصفح الخدمات ويضغط "اطلب الخدمة".
- **النظام:** يسلق الطلب فوراً في Firestore تحت مسمى tickets.

الخطوة التالية؟

نريد أن نجعل نظام التذاكر يرسل تنبيهاً (Notification) لهاتفك الشخصي فور وصول طلب جديد، (هذا يتطلب استخدام Firebase Cloud Functions، وهو أمر متقدم جداً ومبهر للعملاء).

هذا هو المستوى الذي يفرق بين "تطبيق عادي" وبين "نظام إدارة أعمال احترافي". إرسال التنبيهات يضمن أنك لن تفوت أي فرصة عمل حتى لو كنت خارج التطبيق. بما أنك (Push Notifications) تستخدم Firebase بالفعل، سنستخدم (FCM) Firebase Cloud Messaging.

خارطة الطريق التقنية للتنبيهات:

بما أننا في مرحلة التطوير السريع، سنعتمد أسلوب "Firebase Cloud Functions" (وهي كود يُنفذ على السيرفر تلقائياً عند إضافة تذكرة جديدة).

1. إعداد الـ Cloud Function (على مستوى السيرفر)

هذا الكود هو "المراقب"؛ بمجرد أن تضاف تذكرة جديدة في Firestore باسم tickets، يقوم هو بإرسال التنبيه لهاتفك. (ملاحظة: يتم كتابة هذا الكود في واجهة Firebase Functions بـ Node.js أو Python):
رقم الكود JavaS - Code 37

2. في تطبيقك (الجانب البرمجي)

يجب أن يشترك تطبيقك في "موضوع" (Topic) التنبيهات الخاص بالإدارة. أضف هذا الكود في الـ MainActivity عند تشغيل التطبيق لأول مرة

```
Kotlin
import com.google.firebase.messaging.FirebaseMessaging
// عند فتح التطبيق كمدير:
FirebaseMessaging.getInstance().subscribeToTopic("admin_notifications")
    .addOnCompleteListener { task ->
        if (task.isSuccessful) {
            println("تم الاشتراك في تنبيهات الإدارة بنجاح")
        }
    }
}
```

3. متطلبات العمل (عشان ما يوقف التطبيق):

لكي تصل التنبيهات فعلياً، يحتاج تطبيقك إلى Service للتعامل مع الـ `FirebaseMessagingService`. أضف هذا في `AndroidManifest.xml` داخل وسم الـ `<application>`:

```
XML
<service
    android:name=".MyFirebaseMessagingService"
    android:exported="false">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>
```

أنشئ كلاس `MyFirebaseMessagingService` يرث من `FirebaseMessagingService` للتعامل مع استلام التنبيهات في الخلفية.

استراتيجية "المطور التقنية":

- بما أنك صاحب الشركة، التنبيهات هي شريان الحياة.
 - **تنبيه للمدير:** (أنت) يصلك تنبيه فور وجود تذكرة جديدة.
 - **تنبيه للعميل:** (اختياري) يمكنك إضافة كود يرسل تنبيهاً للعميل عندما تقوم أنت بتغيير حالة الطلب في الـ `Firestore` من "Pending" إلى "Completed".
- خطوة البدء بـ لوحة تحكم المدير (Admin Dashboard) يمنحك السيطرة الكاملة، ثم نختم بـ لربط التنبيهات `MyFirebaseMessagingService`.

أولاً: لوحة تحكم المدير (Admin Dashboard)

سننشئ شاشة تظهر لك قائمة بكل التذاكر، مع إمكانية تحديث الحالة (مثلاً من Pending إلى Completed).

1. تحديث ال ViewModel

سنحتاج لدالة لجلب التذاكر ودالة لتحديث حالتها في Firestore:
رقم الكود - Kotlin - Code 38

2. تصميم واجهة ال Admin (AdminScreen.kt)

سنجعلها بسيطة وعملية (بطاقة لكل تذكرة مع زر إجراء):
رقم الكود - Kotlin - Code 39

ثانياً: كلاس MyFirebaseMessagingService

هذا الكلاس هو "المستقبل الذكي" للتنبيهات على هاتفك. بمجرد أن يرسل ال Cloud Function تنبيهاً، سيعمل هذا الكود ليظهر التنبيه للمستخدم (أنت).
رقم الكود - Kotlin - Code 40

لماذا هذا التنظيم يجعلك "مديراً تقنياً ناجحاً"؟

• **الاستقلالية:** لا تحتاج لتعديل كود أو استخدام كمبيوتر لإدارة عملك؛ كل شيء من هاتفك.

• **الاحترافية:** العميل يشعر أن هناك نظاماً يتابعه (حالة الطلب تتحول لمكتمل).

• **الاستجابة:** التنبيهات تضمن أنك "دائماً في الخدمة".

هل انتهيت من ربط ال AdminScreen وال MessagingService؟ بعد إنجاز هذه الخطوة، ستكون شركتك تمتلك "محرك عمليات" (Operations Engine) متكاملًا.

بما أنك أتقنت هذه الجزئية، هل تود أن نضع لمسة أخيرة "أمنية"؟ (منع الوصول لـ باستخدام إيميلك الشخصي كمدير؟) هذا سيجعل Login إلا إذا قمت بعمل AdminTicketScreen النظام آمناً تماماً للاستخدام الفعلي.

تطبيق احترافي بدون نظام أمان هو كـ "بوابة مفتوحة بلا قفل". سنستخدم Firebase فهي الطريقة الأضمن للتأكد من أنك أنت الوحيد (كمدير) من يستطيع الوصول إلى Authentication، شاشة الطلبات (AdminTicketScreen).

1. تفعيل Authentication في Firebase

1. اذهب إلى Firebase Console.

2. من القائمة الجانبية، اختر Authentication.

3. اضغط على Get Started.

4. فعل خيار Email/Password.

2. إضافة المكتبة في المشروع (build.gradle.kts)

رقم الكود Kotli - 41 Code

4. قفل شاشة الـ Admin (Route Guard)

هذا هو "القفل الأمني". في الـ Navigation الخاص بالتطبيق، سنقوم بالتحقق: إذا لم يكن هناك مستخدم مسجل دخول، يتم تحويلك لشاشة الدخول.

```
Kotlin
val currentUser = FirebaseAuth.getInstance().currentUser
if (currentUser != null && currentUser.email == "your_email@example.com") {
    // أنت المدير، مسموح لك بالدخول
    AdminTicketScreen(viewModel)
} else {
    // أنت لست المدير أو لم تسجل دخولك
    LoginScreen(onLoginSuccess = { /* إعادة توجيهه لشاشة الأدمين */ })
}
```

لمسة المطور الإضافية (Security Hardening):

لقد قمنا بإضافة شرط `currentUser.email == "your_email@example.com"`. هذا يضمن أنك أنت فقط من يملك صلاحيات المدير، حتى لو نجح شخص آخر في تسجيل الدخول بإيميل مختلف، لن يفتح التطبيق واجهة التحكم له.

هل أنت جاهز الآن لـ "نقطة النهاية"؟

لقد قمنا ببناء:

- قاعدة بيانات سحابية (Firestore).
- واجهة عميل تفاعلية.
- لوحة تحكم مدير بـ صلاحيات.
- نظام تنبيهات (Push Notifications).
- نظام أمان (Authentication).

أنت الآن تمتلك نظاماً تقنياً متكاملًا. و الآن سنتحدث عن كيفية "تجهيز التطبيق للعمل الفعلي" (Deployment Checklist).

الوصول إلى هذه المرحلة يعني أنك انتقلت من "مبرمج تطبيقات" إلى "مؤسس تقني" (Technical Founder)، يتطلب لمسات أخيرة تضمن الاستقرار (Deployment) تجهيز التطبيق للعمل الفعلي. الأمان، وتجربة المستخدم النهائية.

إليك قائمة المهام الذهبية (Deployment Checklist) لتجهيز تطبيقك للعمل:

1. الإعدادات النهائية (Production Configuration)

- **تغيير مفاتيح الـ API:** تأكد من إزالة أي مفاتيح تجريبية (Debug Keys) واستبدالها بمفاتيح الإصدار النهائي (Release Keys).
- **تعطيل الـ Logs:** في ملف `build.gradle.kts`، تأكد من أن `debuggable` مضبوط على `false` في نسخة الـ `release`.
- **تحديث أيقونة التطبيق (App Icon):** لا تستخدم أيقونة أندرويد الافتراضية! استخدم `Image` ليكون هو الواجهة (Logo) لرفع شعار شركتك في `Android Studio` في `Asset Studio`.
- **اسم التطبيق:** تأكد من ضبط اسم التطبيق باللغتين العربية والإنجليزية في ملف `strings.xml`.

2. الأمان (Security Audit)

- **قواعد (Firebase (Security Rules):** هذه هي أهم خطوة. لا تترك قاعدة البيانات مفتوحة للجميع. اذهب إلى `Firestore Rules` واضبطها بحيث: القراءة: مسموحة للجميع (للاطلاع على الخدمات). الكتابة: مسموحة فقط للمدير (`authenticated admin`) أو لإضافة "تذكرة" جديدة (للمستخدمين).
- **ProGuard/R8:** تأكد من تفعيل "التشويش" (`Obfuscation`) في ملف `build.gradle.kts` لجعل كودك صعب القراءة والسرقة من قبل المنافسين.

3. تجربة المستخدم (User Experience - Final Polish)

- **شاشة الترحيب (Splash Screen):** أضف شاشة ترحيب بسيطة تظهر شعار شركتك أثناء تحميل التطبيق للبيانات.
- **التعامل مع انقطاع الإنترنت:** تأكد من وجود رسالة لطيفة للمستخدم مثل "عذراً، يرجى التحقق من اتصال الإنترنت" بدلاً من إغلاق التطبيق فجأة.

4. الإطلاق (The Launch)

- **فحص النسخة (Final APK/AAB):** قم بتثبيت نسخة الـ release على جهاز حقيقي (وليس محاكي) وجرب كل السيناريوهات: تسجيل الدخول، إضافة خدمة، طلب تذكرة، استلام التنبيه.
- **الرفع إلى Google Play:** بما أنك تملك الآن ملف الـ AAB الموقّع بملف Keystore الخاص بك، اذهب إلى Google Play Console وابدأ عملية إنشاء تطبيق جديد.

نصيحة الخبير للمستقبل (Scaling):

- بمجرد أن يبدأ العملاء باستخدام التطبيق، ستبدأ في تلقي ملاحظاتهم (Feedback). نصيحة: لا تبالغ في إضافة الميزات. حافظ على بساطة التطبيق (Minimalism).
- التالي: بما أنك مهتم بالجانب التسويقي والتقني، ابدأ بوضع رابط تحميل التطبيق في صفحتك على أو موقعك الإلكتروني ليصبح جزءاً من أدوات عمل شركتك LinkedIn

خطواتك الأخيرة الآن: "إجراءات الرفع" (Final Deployment Checklist)

- قبل أن تضغط على "Upload" في Google Play Console، تأكد من أنك قمت بهذه الخطوات الأخيرة:
- **Generate Signed Bundle:** في Android Studio: Build > Generate Signed Bundle / APK اختر Android App Bundle. استخدم نفس الـ KeyStore الذي أنشأته سابقاً (احفظه في مكان آمن جداً، فقدانه يعني عدم قدرتك على تحديث التطبيق لاحقاً).
- **App Store Listing:** جهز لقطات شاشة (Screenshots) احترافية لشاشات التطبيق (شاشة الخدمات، شاشة الطلب، ولوحة التحكم). اكتب وصفاً تسويقياً (Short & Full Description) يركز على القيمة التي ستقدمها للعميل.
- **Content Rating:** في الـ Console، أكمل استبيان "تقييم المحتوى" (Content Rating) وهو سهل جداً لتطبيق خدمات تقنية.
- **Target Audience:** حدد أن الفئة المستهدفة هي المهنيين والشركات (Professional/Business).

كلمة ختامية قبل الإطلاق:

- بصفتك مديراً لشركة تقنية، فإن الإلمام بالتكاليف والمتطلبات الإدارية لـ Google Play Console هو جزء أساسي من إستراتيجيتك التجارية. إليك الدليل الشامل لعملية الإطلاق:

أولاً: الشروط والمتطلبات (قبل البدء)

- **حساب مطور (Google Developer Account):** يجب أن تملك حساب Google شخصي أو خاص بالشركة (يُفضل استخدام إيميل رسمي للشركة).
- **إثبات الهوية:** تطلب جوجل الآن التحقق من الهوية (ID Verification) لصاحب الحساب لضمان الأمان.
- **سياسات المحتوى:** يجب أن يتوافق تطبيقك مع سياسات جوجل (لا محتوى عنيف، لا انتهاك لخصوصية، ولا برمجيات ضارة).
- **توفير بيانات الأمان:** يجب عليك ملء "نموذج أمان البيانات" (Data Safety Form) في الـ Console، حيث توضح فيه ما هي البيانات التي يجمعها تطبيقك (مثل الإيميلات أو معرفات Console، الأجهزة) وكيف يتم تشفيرها.

ثانياً: خطوات رفع التطبيق

- **دفع الرسوم:** 25 دولار أمريكي تدفع لمرة واحدة مدى الحياة. مهما كان عدد التطبيقات في متجرك على Google Play Console.
- **إعداد ملف الـ AAB:** تأكد من رفع ملف بصيغة (Android App Bundle) .aab وليس .apk.
- **وصف التطبيق (Store Listing):**
 - العنوان: اسم التطبيق (بحد أقصى 30 حرفاً).
 - الوصف القصير: جملة تسويقية (80 حرفاً).
 - الوصف الكامل: شرح تفصيلي لخدماتك (حتى 4000 حرف).
 - الأيقونة (Icon): بحجم 512×512 بكسل.
 - صور الشاشة (Screenshots): صور توضح واجهة التطبيق (الخدمات، لوحة التحكم، إلخ).
- **تقييم المحتوى:** الإجابة على استبيان بسيط لتحديد الفئة العمرية المناسبة للتطبيق.
- **المراجعة:** بعد الضغط على "Start Rollout to Production"، ستدخل نسخة تطبيقك في مرحلة مراجعة من قبل فريق جوجل (تستغرق عادة من 3 إلى 7 أيام عمل).

نصيحة استراتيجية لك:

- بما أنك تطلق تطبيق "Services App" كواجهة لشركتك، احرص على:
 - استخدام بريد إلكتروني رسمي: (مثلاً: admin@yourcompany.com) بدلاً من إيميل شخصي بسيط، فهذا يعطي ثقة أكبر للعملاء عند رؤية مطور التطبيق.
 - ربط حسابات التواصل الاجتماعي: جوجل تحب التطبيقات التي تضع روابط لسياسة الخصوصية وموقع إلكتروني خاص الذي تديره.

و بالتوفيق،

هيثم زيدان

08 July 2026

<https://www.patreon.com/haythamzeidan>

Android Full-stack Developer - Lab 4

بصفتك مطور ولديك خبرة في البرمجة الهيكلية، فإن الانتقال إلى إطار عمل متعدد المنصات هو خطوة استراتيجية لتوحيد جهودك. سنتقل كل المفاهيم التي بنيناها في (Cross-Platform) الذي أراه الأكثر استقراراً وقوة) Flutter إلى (Flutter, Authentication, Real-time Updates) لمشاريعك التقنية).

إليك "خارطة الطريق التقنية" لنقل مفاهيمنا إلى Flutter:

1. الإعداد الأساسي (Unified Base)

في Flutter، نستخدم `firebase_core` و `cloud_firestore` بدلاً من كلاسات Kotlin، نستخدم Dart.

إضافة المكتبات (`pubspec.yaml`):

```
dependencies:
  firebase_core: latest
  cloud_firestore: latest
  firebase_auth: latest
  flutter_bloc: latest # لإدارة الحالة (State Management)
```

2. محرك المراقبة (The Monitoring Engine)

بدلاً من `SnapshotListener` في كوتلن، نستخدم `StreamBuilder` في Flutter. هذه هي "الجوهرية" التي تجعل التطبيق ذكياً ولحظياً على أندرويد و iOS في آن واحد:

```
// [رقم الكود 42 Code]
```

3. الفرق الجوهرية في التطوير (Android vs iOS)

بما أنك تستهدف النظامين، يجب أن تنتبه للنقاط التالية:

- **iOS Integration:** عند ربط Firebase، ستحتاج لملف `GoogleService-Info.plist` (بدلاً من `json` في أندرويد) وتعديل `Podfile` ليتوافق مع مكتبات Apple.

- **التصميم:** (لـ iOS) للـ Cupertino و (للأندرويد) Material Design يستخدم Flutter استخدام مكتبات مثل `adaptive_components` لجعل زر "طلب الخدمة" يبدو طبيعياً على النظامين.
- **Permissions:** وونات الكاميرا والموقع الجغرافي (التي بحثت عنها سابقاً) ستحتاج لتعديل ملف `Info.plist` في iOS، تماماً كما عدلت `AndroidManifest.xml` في أندرويد.

4. لماذا Flutter تحديداً لمسارك التقني؟

- **سرعة التطوير:** كود واحد، ومجتمع ضخم جداً.
- **الأداء العالي:** هو ما سيمنح تطبيقك أداءً `Native Machine Code`، يترجم الكود إلى Flutter مبهرراً (خاصة في عمليات المزامنة اللحظية).
- **التوافق مع Hybrid Admin:** بما أنك `configure Graylog` و `Active Directory`، فإن Flutter هو الأفضل في بناء أدوات الـ `System Monitoring` لأن لديه مكتبات قوية للتعامل مع الـ `JSON` والـ `API` المعقدة.

خطوتك القادمة كمهندس (Action Plan):

هل تريد أن نكتب "الهيكل التوجيهي" (Architecture) للمشروع بـ Flutter؟ (مثل تقسيم المشروع إلى `Data Layer` لجلب بيانات الـ `Firebase`، و `UI Layer` لعرضها). أم نركز على كيفية نقل كود `Dart` من كوتلن إلى `MessagingService` و `Admin Dashboard`؟

أقترح البدء بـ الهيكل التنظيمي (Clean Architecture) ليصبح مشروعك قابلاً للتوسع (Scalable) في المستقبل ليدعم iOS و Android دون الحاجة لكتابة كود منفصل لكل نظام.

بما أننا سننتقل إلى Flutter ونبنى Clean Architecture (العمارة النظيفة)، فهذا يضمن أن الكود الذي تكتبه اليوم سيعمل بكفاءة على Android و iOS لسنوات، ويسهل عليك إضافة ميزات جديدة لاحقاً دون أن تنهار البنية التحتية.

إليك هيكل الملفات الاحترافي الذي نعتمد عليه في المشاريع التقنية الكبرى:

1. بنية المجلدات (Clean Architecture Structure)

سيكون المجلد الرئيسي `lib/` مقسماً كالتالي لضمان الفصل التام بين منطق العمل والواجهات:

```

lib/
├── core/           # المكونات المشتركة (Constants, Theme,
Errors)
├── data/
│   ├── models/    # Dart إلى كلاسات JSON تحويل بيانات الـ
│   ├── repositories/ # Firebase (Firestore/Auth) تنفيذ الاتصال بـ
│   └── datasources/ # جلب البيانات الخام من السحابة
├── domain/
│   ├── entities/  # (Ticket, Service) كلاسات الأعمال
│   └── repositories/ # (Interfaces) واجهات الاتصال
└── presentation/
    ├── screens/   # (AdminScreen, واجهات المستخدم
CustomerScreen)
    └── bloc/      # الـ (State Management - إدارة الحالة
الخاص بك StateFlow)

```

2. تعريف الكيان الأساسي (Domain Entity)

في هذا الهيكل، نبدأ بالـ Entity. هذا الكود يعمل على كلا النظامين:

```

// domain/entities/ticket.dart
class Ticket {
  final String id;
  final String serviceName;
  final String customerName;
  final String status;

  Ticket({required this.id, required this.serviceName, required
this.customerName, required this.status});
}

```

3. طبقة البيانات (Data Layer - جلب البيانات)

هنا نستخدم Firestore لجلب البيانات، ونقوم بـ "تحويلها" (Mapping) لتناسب هيكل الـ Entity:

// [رقم الكود 43 Code]

4. لماذا هذا الهيكل هو الأنسب لك؟

- **سهولة الصيانة:** قررت مستقبلاً تغيير قاعدة البيانات من Firebase إلى API خاص بشركتك، ستغير فقط `data/` تلمس أي كود في واجهات المستخدم `presentation/()`
- **التوافق:** هذا الكود مستقل تماماً عن المنصة؛ لا توجد أي إشارة لـ Android أو iOS، وهذا هو سر "كتابة الكود مرة واحدة وتشغيله في كل مكان".
- **قابلية التوسع:** عندما نقوم بربط الـ `MessagingService` لاحقاً، سنضيف فقط مودول جديد في `data/` وسيرتبط فوراً ببقية النظام.

خطوتك العملية الآن:

لكي ننتقل من "التخطيط" إلى "التنفيذ"، أقترح أن نبدأ بإنشاء `Data Model` للـ `Ticket` و الـ `Service` مع دوال الـ `fromMap` و `toMap` (لأنها حجر الأساس في التعامل مع Firebase في Flutter).

1. موديل الخدمة (Service Model)

هذا الموديل يمثل الخدمات التي تعرضها في تطبيقك:

// [رقم الكود 44 Code]

2. موديل التذكرة (Ticket Model)

هذا الموديل يمثل طلب العميل، وهو الذي سيحتوي على الـ `timestamp` لمراقبة الأداء:

// [رقم الكود 45 Code]

ملاحظة تقنية: لقد استخدمنا `factory constructor`، وهو الأسلوب الأكثر احترافية في Dart لجعل الكود نظيفاً وسهل القراءة.

الآن وقد أصبح لدينا الهيكل الأساسي للبيانات، الخطوة التالية هي إنشاء Repository الذي سيربط هذه الموديلات بـ Firebase Firestore. هذا هو "المدير التنفيذي" لطبقة البيانات.

1. ربط البيانات بواجهة المستخدم (UI Layer)

سنستخدم StreamBuilder في واجهة المستخدم. هو الأداة المثالية لأنه يراقب قاعدة البيانات بشكل دائم ويحدث الشاشة فور تغيير أي شيء.

```
// [رقم الكود 46 Code]
```

2. كيف نتعامل مع الأخطاء (Error Handling Strategy)

بصفتك مديراً تقنياً، لا نريد أن يرى العميل "انهيار التطبيق" (App Crash). يجب أن نعتمد استراتيجية "التعامل الاستباقي".

أ. استخدام try-catch في الـ Repository

لا تترك الـ submitTicket عرضة للانهيار إذا انقطع الإنترنت:

```
Future<void> submitTicket(TicketModel ticket) async {
  try {
    await _db.collection('tickets').add(ticket.toMap());
  } on FirebaseException catch (e) {
    // هنا نسجل الخطأ في نظام مراقبتك (مثل Crashlytics)
    print("Firebase Error: ${e.message}");
    throw Exception("فشل الاتصال بالسيرفر، يرجى المحاولة لاحقاً");
  } catch (e) {
    throw Exception("خطأ غير متوقع: $e");
  }
}
```

ب. عرض رسائل للمستخدم (UI Feedback)

```
void _handleTicketSubmission(TicketModel ticket) async {  
  try {  
    await _repository.submitTicket(ticket);  
    ScaffoldMessenger.of(context).showSnackBar(SnackBar(content:  
Text("تم إرسال الطلب بنجاح"))));  
  } catch (e) {  
    ScaffoldMessenger.of(context).showSnackBar(SnackBar(content:  
Text(e.toString())));  
  }  
}
```

ميزة "رفع الصور والمرفقات"

ستنقل تطبيقك من مجرد "نظام طلبات نصي" إلى "نظام إدارة تقني متكامل". تخيل أن العميل يصور عطل الكاميرا أو مشكلة في شبكة الألياف البصرية ويرسلها مباشرة من التطبيق، هذا يوفر عليك وقت التشخيص بنسبة 80%.

1. إعداد Firebase Storage

في Firebase Console، اذهب إلى Storage وقم بتفعيله. تأكد من قواعد الأمان:

```
allow write: if request.auth != null;  
allow read: if true;
```

2. إضافة المكتبة (pubspec.yaml)

```
dependencies:  
  firebase_storage: latest  
  image_picker: latest # اختيار الصور من استوديو الهاتف
```

3. كود رفع الصورة (Repository Layer)

```
// [رقم الكود 47 Code]
```

4. الربط في واجهة المستخدم (UI Layer)

```
// [رقم الكود 48 Code]
```

لمستك الإضافية (Optimization) فإن سرعة التطبيق، استخدم مكتبة مثل `flutter_image_compress` لضغط الصورة قبل الرفع، مما يوفر على العميل استهلاك بيانات الإنترنت ويجعل الرفع فورياً.

نظام المحادثة الفورية (Real-time Chat)

هذا هو الانتقال النوعي من "تطبيق تذاكر" إلى "منصة تواصل تفاعلية".

1. هيكلية الـ Firestore للمحادثة

```
tickets (Collection)
├── ticket_id (Document)
│   ├── messages (Sub-collection)
│   │   └── message_id (Document)
│   │       ├── senderId: "customer" أو "admin"
│   │       ├── text: "...
│   │       └── timestamp: serverTimestamp()
```

2. كود إرسال الرسالة (Data Layer)

```
// lib/data/repositories/chat_repository.dart
Future<void> sendMessage(String ticketId, String text, String
senderId) async {
  await _db.collection('tickets').doc(ticketId)
    .collection('messages')
    .add({
      'text': text,
      'senderId': senderId,
      'timestamp': FieldValue.serverTimestamp(),
    });
}
```

3. شاشة المحادثة (UI Layer) - "الجوهرة الذكية"

// [رقم الكود 49 Code]

شاشة المحادثة المدمجة (Chat Screen) والردود السريعة

1. شاشة المحادثة المدمجة

// [رقم الكود 50 Code]

2. ميزة "الردود السريعة" الذكية (Quick Replies Bar)

شريط أفقي فوق حقل الإدخال يحتوي على الردود الأكثر استخداماً.

// [رقم الكود 51 Code]

مرحلة الإطلاق:

إليك الدليل التقني الشامل لإعداد تطبيق منصة Flutter لمتجري Google Play و App Store.

أولاً: تجهيز بيئة Flutter للـ Release

- تحديث الأيقونة والاسم: استخدم مكتبة flutter_launcher_icons.
- تغيير اسم التطبيق وحزمته: تأكد من تغيير bundleID و applicationId.
- إعداد ملفات الإطلاق (Signing): إنشاء Keystore للأندرويد، وجهاز Mac لتوقيع التطبيق للـ iOS.

ثانياً: خطوات رفع التطبيق على Google Play Console

```
flutter build appbundle --release
```

ارفع ملف الـ aab في Google Play Console.

ثالثاً: خطوات رفع التطبيق على App Store Connect

يتطلب اشتراكاً سنوياً. يجب بناء التطبيق عبر Xcode باستخدام Product > Archive.

رابعاً: الفروقات الجوهرية (نصيحة للمدير التقني)

وجه المقارنة	(أندرويد) Google Play	App Store (iOS)
رسوم التسجيل	25 دولار (مرة واحدة)	99 دولار (سنوياً)
وقت المراجعة	3-7 أيام عمل	24-48 ساعة (أسرع غالباً)
الجهاز المطلوب	أي نظام (ويندوز، ماك، لينكس)	يجب وجود Mac للرفع
الصرامة	متوسطة	عالية جداً (تركز على التصميم)

الكاتب: هيثم زيدان | التاريخ: 08 July 2026

<https://www.patreon.com/haythamzeidan>

مقدمة في إستضافة المواقع الإلكترونية

الدليل الشامل لفهم الاستضافة، الخوادم، النطاقات، والأمان

في عالم الويب، لا يمكن لأي موقع أو تطبيق أن يظهر على الإنترنت دون وجود "مستضيف" (Host). الاستضافة هي المساحة التخزينية والمعالجة التي تشتريها لتخزين ملفات موقعك (مثل الأكواد، الصور، وقواعد البيانات) بحيث تكون متاحة للزوار على مدار الساعة. مع تطور التكنولوجيا، لم تعد الاستضافة تقتصر على خوادم مادية ثابتة، بل ظهرت الاستضافة السحابية لتعيد تشكيل مفهوم مرونة البيانات وكفاءتها.

أنواع الاستضافة

1. الاستضافة التقليدية (Traditional Hosting)

تعتمد الاستضافة التقليدية على تخزين بيانات الموقع على خادم مادي (Physical Server) واحد أو جزء منه. وتنقسم بشكل أساسي إلى ثلاثة أنواع:

- **الاستضافة المشتركة (Shared Hosting):** يتشارك مئات المستخدمين في خادم مادي واحد وموارده (الذاكرة، المعالج، النطاق الترددي). وهي اقتصادية ولكنها تتأثر بضغط الزوار على المواقع الأخرى المجاورة.
- **الخوادم الافتراضية الخاصة (VPS):** يتم تقسيم الخادم المادي الواحد افتراضياً إلى عدة خوادم مستقلة. يحصل كل مستخدم على موارد مخصصة، مما يمنح استقراراً وحرية أكبر من الاستضافة المشتركة.
- **الخوادم المخصصة (Dedicated Servers):** استئجار خادم مادي كامل لصالح موقع واحد فقط. توفر أقصى درجات الأمان والقوة، ولكنها مكلفة وتتطلب إدارة تقنية متقدمة.

2. الاستضافة السحابية (Cloud Hosting)

بدلاً من الاعتماد على جهاز كمبيوتر (خادم) واحد، تعتمد الاستضافة السحابية على شبكة من الخوادم المتصلة معاً (تسمى العنقود أو Cluster). تعمل هذه الخوادم ككيان واحد لتشغيل الموقع.

آلية العمل: إذا تعرض أحد الخوادم في الشبكة لعطل مفاجئ أو ضغط هائل من الزوار، يقوم خادم آخر في الشبكة السحابية بسحب العمل فوراً ومتابعة تشغيل الموقع دون أي توقف (Downtime). يتم توزيع الموارد ديناميكياً حسب حاجة الموقع في نفس اللحظة.

أوجه المقارنة بين الاستضافة التقليدية والسحابية

وجه المقارنة	الاستضافة التقليدية (Shared / VPS / Dedicated)	الاستضافة السحابية (Cloud Hosting)
بنية الخادم	تعتمد على خادم مادي واحد (أو جزء محدد منه).	تعتمد على شبكة افتراضية من عدة خوادم متصلة.
المرونة (Scalability)	محدودة بحدود الخادم المادي. ترقية الخطة تتطلب وقتاً وقد تتسبب في توقف مؤقت.	مرونة فائقة الفورية. يمكنك زيادة الرام أو المعالج بنقرة زر وبشكل تلقائي.
الاعتمادية (Uptime)	إذا تعطل الخادم المادي، يتوقف الموقع تماماً عن العمل.	اعتمادية شبه مطلقة (99.99%). إذا تعطل خادم، يحل محله آخر فوراً.
التكلفة	تكلفة شهرية أو سنوية ثابتة، بغض النظر عن الاستهلاك.	الدفع مقابل الاستهلاك الفعلي (Pay-as-you-go).
الأداء والسرعة	ممتاز إذا كان مخصصاً، ويتراجع في المشتركة بزيادة الضغط.	أداء عالٍ وثابت لتوزيع الأحمال (Load Balancing).

متى تختار كل منهما؟

اختر التقليدية إذا: كان موقعك جديداً/صغيراً، ميزانيتك محدودة، ولا تتوقع قفزات مفاجئة في أعداد الزوار.

اختر السحابية إذا: كنت تدير متجرًا كبيراً، تطبيقاً ذكياً، أو موقعاً يشهد تذبذباً في الزوار وتوقف الموقع يسبب خسائر.

أنواع الاستضافة من حيث نظام التشغيل

تنقسم استضافة المواقع من حيث نظام التشغيل (OS) إلى نوعين رئيسيين يهيمنان على سوق الخوادم: استضافة لينكس (Linux Hosting) واستضافة ويندوز (Windows Hosting).

1. استضافة لينكس (Linux Hosting)

هي الاستضافة الأكثر شيوعاً واستخداماً على الإنترنت، وتعتبر الخيار الافتراضي لمعظم المواقع. تعتمد على أنظمة توزيع مفتوحة المصدر.

- **التكلفة المنخفضة:** مجاني ومفتوح المصدر، تكلفته دائماً أقل.
- **الاستقرار والأمان:** قدرة عالية على التحمل ومقاومة الثغرات.
- **التقنيات المدعومة:** PHP, Python, Node.js, MySQL, WordPress, Joomla.

2. استضافة ويندوز (Windows Hosting)

مخصصة للمواقع والتطبيقات التي تعتمد على البرمجيات والتقنيات المملوكة لشركة مايكروسوفت (Microsoft).

- **التكامل:** الخيار الوحيد لتشغيل بيئات العمل المتقدمة من مايكروسوفت.
- **التقنيات المدعومة:** ASP.NET, Core .NET, MSSQL, Access, IIS.

وجه المقارنة	استضافة لينكس (Linux)	استضافة ويندوز (Windows)
التكلفة	اقتصادية (مفتوح المصدر)	أعلى تكلفة (رسوم تراخيص مايكروسوفت)
لوحة التحكم	cPanel أو DirectAdmin	Plesk
قواعد البيانات	MySQL	MSSQL

ما هو اسم النطاق Domain Name؟

اسم النطاق (Domain Name) هو العنوان الفريد الذي يكتبه المستخدمون في شريط المتصفح (مثل الوصول إلى موقعك الإلكتروني. إذا كان موقعك عبارة عن "منزل"، فإن الاستضافة هي الأرض (www.google.com) واسم النطاق هو "عنوان المنزل".

الأنظمة تتعامل بالأرقام عبر (IP Address). الـ DNS يقوم بتحويل اسم النطاق الذي تحفظه إلى رقم الـ IP الخاص بال خادم.

- **النطاق الفرعي (Subdomain):** الجزء الأول لتنظيم الأقسام (مثل blog).
- **اسم النطاق الأساسي (Domain Name):** يعبر عن مشروعك (example).
- **امتداد النطاق (TLD):** الجزء الأخير يحدد التخصص (.com, .net, .org) أو جغرافيا (.sa, .ps).

ما هي بروتوكولات نقل الملفات؟

هي قواعد تحدد كيفية حزم وإرسال واستقبال البيانات بين الأجهزة والخوادم. أشهرها:

- **FTP:** النظام التقليدي لرفع الملفات، لكنه غير آمن.
- **SFTP:** البديل الآمن والحديث، يشقّر كل البيانات المتبادلة.
- **FTPS:** يضيف طبقة حماية SSL/TLS.
- **HTTPS:** نظام نقل الملفات الآمن عبر المتصفح (للتحميل والرفع).
- **P2P:** مشاركة الملفات بين الأجهزة مباشرة دون خادم مركزي.

ما هو الـ DNS؟

الـ DNS (نظام أسماء النطاقات) هو "دليل هاتف الإنترنت". يترجم أسماء المواقع (google.com) إلى عناوين رقمية (IP Address).

رحلة الطلب:

1. مستعلم الـ DNS المنفذ (DNS Recursor).
2. خادم الجذر (Root Nameserver).
3. خادم نطاق المستوى الأعلى (TLD Nameserver).
4. خادم الأسماء المعتمد (Authoritative Nameserver).

ما هي شهادة الأمان (SSL)؟

تنشئ رابطاً مشفراً بين خادم موقع الويب ومتصفح الزائر، محولة البروتوكول من http إلى https الآمن.

- تحمي البيانات (كأرقام البطاقات وكلمات المرور) من التجسس.
 - تحسن ترتيب الموقع في محرك بحث جوجل (SEO).
 - تزيد من ثقة الزوار وتمنع ظهور تحذير "موقع غير آمن".
- يمكن الحصول عليها مجاناً عبر (Let's Encrypt) لمعظم المواقع، أو شراؤها (DV, OV, EV) للمشاريع الكبرى التي تتطلب تحققاً قانونياً وموثوقية أعلى.

الدورات والشهادات المعتمدة

تنقسم الشهادات حسب التخصص والبيئة التقنية:

- **شهادات لوحات التحكم:** cPanel & WHM Certifications, Plesk Certifications.
- **إدارة الخوادم (Linux):** RHCSA (Red Hat), LFCS, CompTIA Linux+.
- **الاستضافة السحابية:** AWS Cloud Practitioner / SysOps, Azure Administrator (AZ-104), Google Associate Cloud Engineer.
- **إدارة مواقع محددة:** WCA for WordPress.

دليل إختيار قواعد البيانات

اختيار نظام قواعد البيانات الأنسب لعملك يعتمد بشكل أساسي على نوع المشروع الذي تعمل عليه وأسلوب معالجة البيانات. إذا كنت تبحث عن التوجيه الأنسب، إليك الخيارات الأكثر طلباً في السوق حالياً ومجال استخدام كل منها:

1. لقواعد البيانات العلائقية (SQL - Relational Databases)

إذا كان المشروع يعتمد على هيكلية بيانات مرتبطة ببعضها (مثل بيانات المستخدمين، الطلبات، والعمليات المالية)، فهذا هو الخيار الأمثل:

PostgreSQL

يُعتبر اليوم "المعيار الذهبي" لقواعد البيانات مفتوحة المصدر. لماذا تتعلمه؟ يدعم ميزات متقدمة جداً، أداء عالٍ، وقابلية للتوسع، وهو المفضل حالياً لشركات التقنية الكبرى. الاستخدام: تطبيقات الويب والتطبيقات التي تحتاج لتكامل معقد (Complex Queries).

MySQL

الأكثر شهرة وانتشاراً بسبب ارتباطه الوثيق مع WordPress والعديد من منصات الاستضافة. لماذا تتعلمه؟ سهل التعلم، وثائقه متوفرة بكثرة، ومثالي للمشاريع التي تبدأ كمVP (منتج أولي).

2. لقواعد البيانات غير العلائقية (NoSQL)

إذا كان تطبيقك يتطلب مرونة عالية في شكل البيانات، أو يحتاج للتعامل مع كميات ضخمة من البيانات غير المنظمة:

MongoDB

يعتمد على نظام الـ Documents (JSON-like).

لماذا تتعلمه؟ مرونة لا محدودة في تعديل بنية البيانات دون الحاجة لتعديل الجداول (Schema-less)، وهو ممتاز جداً مع بيئات Node.js و React Native.

Firebase (Firestore)

هي خدمة من جوجل (Backend-as-a-Service).

لماذا تتعلمه؟ توفر لك قاعدة بيانات في الوقت الحقيقي (Real-time)، ومصادقة مستخدمين، وتخزين ملفات، وتتكامل بشكل سلس جداً مع Flutter. إذا كنت تريد بناء تطبيق موبايل بسرعة كبيرة دون الانشغال بإدارة الخوادم، فهي الخيار رقم 1.

3. لقواعد البيانات المدمجة (Local Databases)

إذا كنت مطور تطبيقات موبايل، ستحتاج أحياناً لتخزين بيانات على جهاز المستخدم نفسه:

SQLite

المحرك الأكثر استخداماً في العالم (مدمج في كل أنظمة أندرويد و iOS).

Drift (سابقاً Moor)

إذا كنت تستخدم Flutter، فهذه المكتبة هي الأفضل للتعامل مع SQLite بشكل احترافي مع دعم كامل لـ Dart.

نصيحة خبير تقني

• إذا كان هدفك التوظيف في الشركات الكبرى أو بناء نظام خلفي (Backend) متين: تعلم هي مهارة أساسية ستبقى معك لسنوات طويلة وتمنحك فهماً عميقاً لكيفية عمل قواعد PostgreSQL البيانات.

• إذا كان تركيزك هو "سرعة الإنجاز" في تطبيقات الموبايل: ابدأ فوراً بـ Firebase Backend. فهي ستختصر عليك أسابيع من برمجة الـ (Firestore)

• فهم الأساسيات: بغض النظر عن النظام الذي تختاره، تعلم لغة SQL بشكل جيد، لأن مفاهيم (Joins, Indexes, Transactions) هي مفاهيم عالمية ستجدها في كل مكان

الدليل الاحترافي لتطوير وحوكمة التطبيقات الذكية

استراتيجيات الربح المتكاملة وهندسة البنية التحتية عبر خدمات Firebase

أولاً: استراتيجيات الربح من التطبيقات الذكية (App Monetization)

لم يعد الربح من التطبيقات الذكية مجرد خيار جانبي، بل تحول إلى صناعة عالمية تُدر مليارات الدولارات سنوياً. مع التطورات التقنية المتسارعة وتغير سلوك المستخدمين، لم يعد المطورون يعتمدون على أسلوب واحد لتحقيق الدخل، بل أصبح التوجه السائد هو الاعتماد على النماذج الهجينة (Hybrid Models) التي تدمج أكثر من طريقة لتعظيم الأرباح.

1. نماذج استراتيجيات الربح الأساسية (Monetization Models)

الاشتراكات داخل التطبيق (In-App Subscriptions)

يُعد هذا النموذج الحصان الرابح والأكثر استقراراً في الوقت الحالي. يدفع المستخدم رسوماً دورية (أسبوعية، شهرية، أو سنوية) مقابل الاستمرار في استخدام التطبيق أو الوصول إلى محتوى متجدد.

النزعة الحالية: تشير البيانات الحديثة إلى قفزة هائلة في الاشتراكات الأسبوعية، حيث أصبحت تستحوذ على أكثر من 50% من إيرادات تطبيقات الخدمات، لأنها توفر تجربة منخفضة المخاطر للمستخدم في البداية وتزيد من معدل التحويل السريع.

أمثلة: تطبيقات اللياقة البدنية، المنصات التعليمية، تطبيقات تحرير الصور والفيديو المعتمدة على الذكاء الاصطناعي.

الشراء داخل التطبيق (In-App Purchases - IAP)

يتيح هذا النموذج تحميل التطبيق مجاناً، لكنه يوفر مزايا أو أدوات إضافية مدفوعة. وتنقسم هذه المشتريات إلى ثلاثة أنواع:

- **المنتجات الاستهلاكية (Consumables):** مثل العملات الافتراضية في الألعاب، أو نقاط إضافية يتم استهلاكها وإعادة شرائها.
- **المنتجات غير الاستهلاكية (Non-Consumables):** مزايا تشتريها مرة واحدة وتبقى معك للأبد، مثل إزالة الإعلانات نهائياً أو فتح مستوى معين.
- **المحتوى التجميلي (Cosmetics):** تغيير شكل واجهة التطبيق أو شراء ملابس (Skins) للشخصيات داخل الألعاب، وهو اتجاه مريح جداً لأنه يعتمد على رغبة المستخدم في التميز دون التأثير على عدالة اللعب.

الإعلانات داخل التطبيق (In-App Advertising)

الموديل الأسهل للتطبيقات التي تمتلك قاعدة جماهيرية ضخمة ولكن بمعدل إنفاق منخفض. وتشمل عدة أنواع:

- **الإعلانات بمكافأة (Rewarded Ads):** هي الأنجح على الإطلاق؛ حيث يختار المستخدم طوعية مشاهدة إعلان فيديو مدته 30 ثانية مقابل الحصول على ميزة (مثل محاولة إضافية في لعبة أو فتح ميزة مقفلة). تحقق هذه الإعلانات أعلى العوائد (eCPM).
- **الإعلانات البينية (Interstitial Ads):** إعلانات ملء الشاشة تظهر في الفواصل الطبيعية للتطبيق (مثل الانتقال من صفحة إلى أخرى).
- **الإعلانات المدمجة (Native Ads) والبانر (Banners):** إعلانات تندمج مع تصميم التطبيق وتظهر أسفل الشاشة أو بين المقالات.

النموذج المجاني المدفوع (Freemium) والتسويق بالعمولة

تقديم النسخة الأساسية من التطبيق مجاناً لجمع أكبر عدد من المستخدمين، مع وضع الميزات المتقدمة والأدوات الاحترافية خلف "جدار دفع" (Paywall). وفي حالة التطبيقات المتخصصة (كالطهي أو السياحة) يتم دمج التسويق بالعمولة والرعاية عبر تضمين روابط لمنتجات وخدمات لشركات أخرى والحصول على عمولة لكل عملية بيع.

2. النموذج الهجين والخطوات الاستراتيجية

التطبيقات الأكثر ربحية لا تضع بيضها في سلة واحدة، بل تقسم مستخدميها إلى فئات سلوكية (User Segmentation):

- **المستخدم المجاني (Free User):** يتم الريح منه عبر الإعلانات المدمجة والمكافآت.
- **المستخدم المتفاعل (Engaged User):** يتم تحويله إلى نظام الاشتراكات للحصول على تجربة خالية من الإعلانات.
- **المستخدم الشغوف (Power User):** المستعد لدفع مبالغ إضافية لشراء أدوات حصريّة ومحتوى متميز بجانب اشتراكه.

💡 الاستراتيجيات النفسية لتحسين واجهات الدفع (Paywall Optimization)

قاعدة الخيارات الثلاثة تقديم 3 خيارات (أسبوعي، شهري، سنوي) مع إبراز الخيار السنوي بخصم مغرٍ (40% - 60%) لتبدو هي الصفقة الأفضل وتجنب "شلل الاختيار".

التوطين والترجمة (Localization): ترجمة شاشة الدفع وعرض الأسعار بالعملة المحلية يرفع نسب التحويل بمعدلات تتجاوز 60%.

التوقيت الذكي (Contextual Paywalls): إظهار شاشة الدفع فوراً بعد إتمام المستخدم لمهمة ناجحة حيث يكون في أعلى درجات الرضا عن المنتج.

3. أشهر منصات الربح والوساطة

فئة التطبيق الرئيسية	الاستراتيجية الأفضل	المنصات المقترحة
تطبيقات أدوات وخدمات (Utility Apps)	اشتراكات شهري/سنوي ونموذج Freemium	RevenueCat, Adapty, Paddle
ألعاب كاجوال وسريعة (Hyper-Casual Games)	إعلانات بمكافأة وإعلانات بينية مكثفة	AppLovin (MAX), Google AdMob
تطبيقات ألعاب معقدة ومحتوى غني	دمج (إعلانات بمكافأة + مشتريات أدوات تجميلية)	Unity Ads, RevenueCat, Meta Audience Network

🔧 نصيحة المطورين المحترفين

لا تعتمد على شبكة إعلانية واحدة مطلقاً. استخدم AppLovin MAX أو Google AdMob Mediation كمنصة وساطة رئيسية، واجعل بقية الشبكات (مثل Meta و Unity) تتنافس داخلها لتحقيق أعلى عائد ممكن للإعلان الواحد.

ثانياً: منصة Firebase من Google وإدارة البنية التحتية الخلفية

تُعد Firebase منصة خدمات تطوير تطبيقات متكاملة من Google، وتُصنف ضمن فئة "Backend-as-a-Service" (BaaS)، أي أنها توفر للمطورين بنية تحتية خلفية جاهزة تغنيهم عن بناء وإدارة الخوادم (Servers) وقواعد البيانات من الصفر بهدف تسريع عملية التطوير.

1. أبرز خدمات منصة Firebase

- **البناء والتطوير (Build):** تشمل Authentication لمصادقة المستخدمين، وقواعد البيانات السحابية NoSQL مثل Cloud Firestore و Realtime Database، بالإضافة إلى Cloud Storage لتخزين الملفات و Cloud Functions لتشغيل أكواد جانب الخادم.

- **مراقبة جودة الأداء (Release & Monitor):** تشمل أداة Crashlytics لتتبع الأخطاء والانهيئات بشكل لحظي، وأداة Performance Monitoring لمراقبة سرعة التطبيق.
- **زيادة التفاعل (Engage):** تشمل Firebase Cloud Messaging (FCM) لإرسال الإشعارات مجاناً، و Google Analytics لتحليل سلوك المستخدمين.

2. المميزات والعيوب لمنصة Firebase

المميزات	العيوب
سرعة التطوير: توفر الوقت والجهد في بناء الواجهة الخلفية.	الاعتماد الكلي (Vendor Lock-in): يصعب الانتقال لاحقاً لمنصة أخرى.
بنية تحتية سحابية: سهولة تامة في توسيع النطاق (Scaling).	قواعد البيانات NoSQL: ليست الأفضل للعلاقات المعقدة (SQL).
دعم واسع: تتكامل بسلاسة مع Flutter, React Native, Android, iOS.	تكلفة النمو: قد ترتفع التكلفة بشكل كبير مع زيادة المستخدمين.

ثالثاً: الدليل البرمجي والتطبيقي لربط Firebase بإطار عمل Flutter

لربط تطبيق Flutter مع Firebase، نقوم بإعداد الأدوات اللازمة عبر سطر الأوامر ثم تهيئة الحزم البرمجية داخل كود التطبيق.

1. تهيئة وإعداد المشروع (Flutter Initialization)

نقوم بتنفيذ أداة FlutterFire CLI لتوليد ملف الإعدادات تلقائياً من خلال الأوامر التالية:

```
dart pub global activate flutterfire_cli
flutterfire configure
```

في ملف `pubspec.yaml`، نقوم بإضافة المكتبات الأساسية والخدمات المطلوبة:

```
dependencies:
  flutter:
    sdk: flutter
  firebase_core: ^latest_version
  firebase_auth: ^latest_version
  cloud_firestore: ^latest_version
  firebase_storage: ^latest_version
  image_picker: ^latest_version
```

نقوم بتهيئة منصة Firebase داخل ملف **main.dart** قبل تشغيل واجهات التطبيق:

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(MyApp());
}
```

2. تطبيق عملي لخدمة المصادقة (Authentication)

كود برمجي لتسجيل دخول المستخدم باستخدام البريد الإلكتروني وكلمة المرور:

```
import 'package:firebase_auth/firebase_auth.dart';

Future<void> signIn(String email, String password) async {
  try {
    await FirebaseAuth.instance.signInWithEmailAndPassword(
      email: email,
      password: password,
    );
    print("تم تسجيل الدخول بنجاح");
  } on FirebaseAuthException catch (e) {
    print("خطأ: \${e.message}");
  }
}
```

3. إدارة البيانات والملفات (Cloud Firestore & Cloud Storage)

الممارسة القياسية والفضلى لإضافة مستخدم جديد وحفظ بيانه سحابياً في قاعدة البيانات المستندات:

```
import 'package:cloud_firestore/cloud_firestore.dart';

Future<void> addUser(String userId, String name, String downloadUrl) async {
  await FirebaseFirestore.instance.collection('users').doc(userId).set({
    'name': name,
    'profile_image_url': downloadUrl,
    'created_at': FieldValue.serverTimestamp(),
  }).then((value) => print("تمت إضافة المستخدم بنجاح"));
}
```

رفع صورة من استوديو جهاز المستخدم إلى وحدة التخزين السحابية (Cloud Storage) وجلب رابط التحميل:

```

import 'dart:io';
import 'package:firebase_storage/firebase_storage.dart';
import 'package:image_picker/image_picker.dart';

Future<String?> uploadImage() async {
  final ImagePicker picker = ImagePicker();
  final XFile? image = await picker.pickImage(source: ImageSource.gallery);
  if (image == null) return null;

  File file = File(image.path);
  Reference ref = FirebaseStorage.instance.ref().child('user_images/\${
    DateTime.now().toString()}');

  await ref.putFile(file);
  String downloadUrl = await ref.getDownloadURL();
  return downloadUrl;
}

```

ملاحظة أمنية هامة وقواعد الحماية

عند البدء، تكون قواعد الأمان في "وضع الاختبار" (Test Mode) التي تسمح لأي شخص بالقراءة والكتابة. قبل إطلاق التطبيق للمستخدمين، تأكد من الانتقال إلى Firebase Console وضبط قواعد الأمان (Security Rules) لمنع الوصول غير المصرح به، على سبيل المثال

```
allow read, write: if request.auth != null;
```

منصات تطوير التطبيقات الأكثر انتشاراً في العالم العربي

دليل تحليلي شامل للغات والأطر البرمجية الأكثر طلباً في سوق العمل التقني العربي

في العالم العربي، تتماشى طفرة تطوير التطبيقات الذكية بشكل كبير مع التوجهات العالمية، لكن مع تركيز خاص على التقنيات التي توفر سرعة في التنفيذ، كفاءة في التكلفة، ودعم قوي للغة العربية. سواء كنت تفكر في دخول هذا المجال كمهندس برمجيات أو ترغب في إطلاق مشروعك الخاص، إليك اللغات والتقنيات الأكثر انتشاراً وطلباً في السوق العربي.

1. الأطر متعددة المنصات (Cross-Platform) - الأكثر طلباً للمشاريع الناشئة

تحتل هذه الأدوات بشعبية جافرية في العالم العربي (خاصة في مصر، السعودية، والإمارات) لأنها تتيح بناء تطبيق واحد يعمل على أندرويد و iOS في نفس الوقت، مما يوفر نصف وقت وجهد التكلفة الاستثمارية للمشروع.

Dart إطار عمل Flutter

ترفع **Flutter** (المدعوم من Google) على عرش تطوير التطبيقات في العالم العربي خلال السنوات الأخيرة. يفضلها المطورون والشركات الناشئة لسهولة بناء واجهات مستخدم جذابة وسريعة، ودعمه الممتاز والسلس للغة العربية واتجاهات النصوص من اليمين إلى اليسار (RTL).

JavaScript / TypeScript إطار عمل React Native

المدعوم من شركة **Meta**. ما زال خياراً رئيسياً للعديد من الشركات الكبرى والمنصات التي تمتلك بالفعل مواقع إلكترونية مبنية بـ **React**، حيث يسهل نقل الكفاءات البرمجية لتطوير التطبيقات دون الحاجة لإعادة تدريب الفريق على لغات جديدة بالكامل.

2. التطبيقات الأصلية (Native Apps) - للمشاريع الضخمة والأداء العالي

تُستخدم هذه اللغات عندما يتطلب التطبيق أداءً فائقاً، أو تعاملًا مباشراً ومعقداً مع عتاد الهاتف (مثل الحساسات والذكاء الاصطناعي والكاميرا)، وتعتمد عليها البنوك، الشركات الحكومية، والتطبيقات الكبيرة التي لا تقبل المساومة على الأداء.

أ. لنظام أندرويد (Android)

Kotlin اللغة الرسمية لأندرويد

أصبحت اللغة الرسمية والأساسية لتطوير تطبيقات أندرويد. استبدلت لغة Java بشكل كبير في المشاريع الحديثة بفضل اختصارها للكود وقلة أخطائها البرمجية، وهي مطلوبة بشدة في سوق العمل الخليجي والمصري.

Java الأنظمة القديمة

رغم تراجعها الملحوظ أمام Kotlin، إلا أنها لا تزال حية ومستعملة بانتظام في صيانة وتحديث التطبيقات الضخمة والقديمة (Legacy Systems) في البنوك والشركات الكبرى والمؤسسات الخدمية.

ب. لنظام آي أو إس (iOS)

Swift بيئة عمل آبل

اللغة الحصرية والمدعومة من Apple لتطوير تطبيقات iPhone و iPad. تحظى بطلب مرتفع جداً وبأجور مرتفعة في دول الخليج العربي (مثل السعودية والإمارات) نظراً للنسبة العالية لمستخدمي هواتف iPhone هناك مقارنة بباقي الدول العربية.

3. لغات السيرفر والبرمجة الخلفية (Backend) للتطبيقات

أي تطبيق ذكي يحتاج إلى "خلفية" متينة لإدارة البيانات، المستخدمين، الأمان، والعمليات الحسابية السحابية. اللغات الأكثر انتشاراً في السوق العربي لهذا الغرض هي:

PHP إطار عمل Laravel

يحظى **Laravel** بشعبية هائلة في العالم العربي لبناء لوحات التحكم وربط التطبيقات بقواعد البيانات، وذلك لسهولة الملموسة ووفرة المطورين والمصادر التعليمية التي تدعمه.

JavaScript بيئة تشغيل Node.js

مطلوب بشدة للتطبيقات الحية التي تتطلب تحديثات فورية وزمن استجابة فوري (**Real-time**) مثل تطبيقات المحادثة الفورية، ألعاب الأونلاين، وتطبيقات التوصيل التشاركي.

Python Django / FastAPI

يشهد نمواً متصاعداً ومتسارعاً في الآونة الأخيرة، خاصة مع دخول تقنيات الذكاء الاصطناعي، الحوسبة السحابية، وتحليل البيانات كجزء أساسي ومحوري من وظائف التطبيقات الذكية الحديثة.

ملخص التوزيع الجغرافي والتقني في السوق العربي

الفئة التقنية	التقنيات الرائدة	محرك الطلب الأساسي في السوق العربي
Cross-Platform	Flutter, React Native	الشركات الناشئة، توفير التكلفة، السرعة، دعم ممتاز للغة العربية وال RTL.
Native (Android/ iOS)	Kotlin, Swift	المؤسسات الحكومية، البنوك، والشركات الكبرى (طلب مرتفع جداً في الخليج).
Backend APIs	Laravel, Node.js, Django	إدارة البيانات السحابية، لوحات التحكم، وتطبيقات التوصيل الفوري والذكاء الاصطناعي.